

pj1 报告

曹兮 22307140119

github 代码及数据集链接（没注意把数据集一并上传了，非常抱歉）：

https://github.com/22307140119/DATA130011_pj1_22307140119

模型权重链接（一共包含8个模型，性能最好的MLP模型后缀为9834，性能最好的CNN模型后缀为9720）：

<https://pan.baidu.com/s/1MQj3Y7-ZOCF4z2Ukt7yc7g?pwd=9vth>

目录

报告中部分代码思路的描述过于冗长，以及目录是手动添加不支持自动跳转，再一次非常抱歉。

- 0. 线性层和交叉熵损失
 - 0.1 op.py: 线性层 Linear
 - 0.2 交叉熵损失 MultiCrossEntropyLoss
- 1. 修改网络结构
- 2. 修改 lr_scheduler.py，实现不同学习率策略
 - 2.1 lr_scheduler.py: MultiStepLR
 - 2.2 lr_scheduler.py: ExponentialLR
- 3. 尝试 L2 正则化
 - 3.1 optimizer.py: SGD 中已实现 L2 正则化
 - 3.2 op.py: L2Regularization
- 4. 实现 CNN
 - 4.1 op.py: conv2D
 - 4.1.1 conv2D: 用四层嵌套循环，效率低
 - 4.1.2 conv2D: 向量化后的版本，效率较高
 - 4.2 op.py: MaxPooling
 - 4.2.1 MaxPooling: 和 conv2D 类似，用四层嵌套循环
 - 4.2.2 MaxPooling: 向量化后错误的实现，但在大数据集上的效果更好
 - 4.2.3 MaxPooling: 正确的向量化实现
 - 4.3 op.py: Flatten
 - 4.4 models.py: 卷积神经网络 Model_CNN 类
 - 4.5 其它适配
 - 4.5.1 runner.py
 - 4.5.2 plot.py
 - 4.5.3 test_train_CNN.py 和 test_model_CNN.py
- 5. Moment GD 优化器
 - 5.1 Moment GD 实现
 - 5.1.1 optimizer.py: MomentGD
 - 5.1.2 op.py: Kaiming 初始化
 - 5.1.3 models.py: 支持创建 MLP 时指定参数初始化方式

- 5.2 训练 MLP
 - 5.2.1 MLP: Moment GD + Kaiming
 - 5.2.2 MLP: Moment GD + StdNormal
 - 5.2.3 MLP: SGD + Kaiming
 - 5.2.4 MLP: SGD + StdNormal
- 5.3 训练 CNN
 - 5.3.1 CNN: Moment GD + Kaiming + Kaiming
 - 5.3.2 CNN: SGD + Kaiming + Kaiming
 - 5.3.3 CNN: SGD + Kaiming + StdNormal
- 6. 数据增强: data_augmentation.py
 - 6.1 较为底层的实现
 - 6.1.1 双线性插值
 - 6.1.2 图像的批量平移
 - 6.1.3 图像的批量旋转
 - 6.1.4 图像的批量缩放
 - 6.1.5 其它适配
 - 6.1.6 效果可视化
 - 6.2 "数据增强器"类 MyDataAugmentor
 - 6.3 静态数据增强
 - 6.4 动态数据增强
 - 6.4.1 修改 runner.py: RunnerM 类
 - 6.4.2 修改 test_train_CNN.py
 - 6.5 训练 MLP
 - 6.5.1 MLP: 使用"静态数据增强"
 - 6.5.2 MLP: 使用"动态数据增强"
 - 6.6 训练 CNN
 - 6.6.1 CNN: 使用"静态数据增强"
 - 6.6.2 CNN: 使用"动态数据增强"
- 7. 正则化: Dropout
 - 7.1 Dropout 实现
 - 7.1.1 修改 models.py
 - 7.1.2 其它修改
 - 7.2 应用 dropout 训练 MLP
 - 7.3 应用 dropout 训练 CNN
- 8. 可视化权重
 - 8.1 可视化 MLP 权重
 - 8.1.1 MLP 第一层全连接层的权重整体可视化
 - 8.1.2 MLP 第一层全连接层的权重以矩阵形式可视化
 - 8.1.3 MLP 第二层全连接层的权重整体可视化
 - 8.2 可视化 CNN 权重
 - 8.2.1 CNN 卷积层1权重可视化
 - 8.2.2 CNN 图片作用卷积层1后的特征可视化
 - 8.2.3 CNN 图片作用激活函数1和池化层1后的特征可视化
 - 8.2.4 CNN 卷积层2权重可视化
 - 8.2.5 CNN 图片作用卷积层2、激活函数2和池化层2后的特征可视化

- 8.2.6 CNN 全连接层1权重可视化
- 8.2.7 CNN 全连接层2权重可视化

0. 线性层和交叉熵损失

0.1 op.py: 线性层 Linear

`Linear(in_dim, out_dim, weight_decay, weight_decay_lambda)`

- 前向传播 `forward(self, X)`
 - 由于 `optimizer` 中更新的是字典 `self.params` 中的参数，在前向传播之前需要先把 `self.params` 中的参数取出来。
 - 返回 $XW+b$ 。
- 反向传播 `backward(self, grad)`
 - 计算 $d(XW+b) / dX$ 并返回，用于反向传播。
 - 计算 $d(XW+b) / dW$ 和 $d(XW+b) / db$ ，存入 `self.grads` 字典中，用于参数更新。

0.2 op.py: 交叉熵损失 MultiCrossEntropyLoss

`MultiCrossEntropyLoss(model, max_classes)`

- 初始化 `__init__(self, model = None, max_classes = 10)`
 - 调用父类构造函数，同时初始化一些必要的变量
 - 初始化 `self.has_softmax` 为 `True`
 - 初始化 `self.grads` 用于存储梯度（作为参数传播到线性层的 `backward()` 中）
- 前向传播 `forward(self, predicts, labels)`
 - `labels` 存入 `self.labels` 供反向传播使用。
 - 根据 `self.has_softmax` 的取值，
 - 如果为 `True` 则先对输入作用 `softmax()` 再存入 `self.probs` 变量。
 - 如果为 `false` 则直接将输入存入 `self.probs` 变量。
 - 再对 `self.probs` 变量作用交叉熵损失：
 - 根据 `labels` 的值，从 `self.probs` 变量中选出需要被取 `log` 的值 `correct_probs`（即“正确”的类别对应的概率值）。
 - 对 `correct_probs` 中的每个值取对数后再求平均得到损失值 `loss`，这里在取对数前 $+ 1e-15$ 防止出现 `log(0)` 的情况。
- 反向传播 `backward(self)`
 - 先把提供的 `labels` 转化为 one-hot 编码（后面求导用）。
 - 再根据 `self.has_softmax` 的取值分别完成求导，导数值存入 `self.grads` 中。

修改 `lr_scheduler` 为 `StepLR`，其余保持原有设置不变：

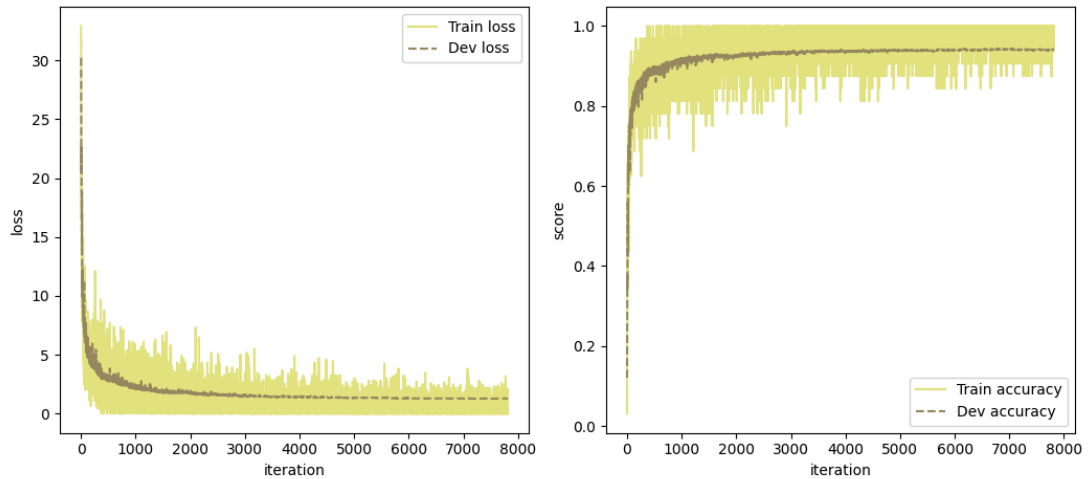
```
linear_model = nn.models.Model_MLP([train_imgs.shape[-1], 600, 10],
'ReLU', [1e-4, 1e-4])
optimizer = nn.optimizer.SGD(init_lr=0.06, model=linear_model)
scheduler = nn.lr_scheduler.StepLR(optimizer=optimizer, step_size=400,
gamma=0.9)
```

```

loss_fn = nn.op.MultiCrossEntropyLoss(model=linear_model,
max_classes=train_labs.max()+1)
runner = nn.runner.RunnerM(linear_model, optimizer, nn.metric.accuracy,
loss_fn, scheduler=scheduler)
runner.train([train_imgs, train_labs], [valid_imgs, valid_labs],
num_epochs=5, log_iters=100, save_dir=r'./best_models')

```

其余部分仍然使用给定的代码，训练结果：



训练集和验证集上的准确率约为 0.94。

1. 修改网络结构，使用 nHidden 向量（列表）指定隐藏层的神经元数量。

原本的代码中，在创建模型 `linear_model` 时，传入的参数 `[train_imgs.shape[-1], 600, 10]` 即指定了各个层的神经元个数。

但是隐藏层的神经元数量 600 被包含在了 `size_list` 参数中，而不是单独由一个列表指定。

这里需要用 `nHidden` 向量指定隐藏层的神经元数量，需要修改 `models.py` 中根据提供的各层神经元数量创建模型的 `__init__()` 初始化函数。

- 原本 `models.py __init__()` 的逻辑。
 - 遍历 `size_list`，根据 `size_list` 中的值创建对应维数的线性层。
 - `size_list` 把 输入层维数、隐藏层维数、输出层维数 全都存储在一个列表中。
 - 直接遍历就可以完成所有线性层的创建。
- 修改 `models.py __init__()` 的逻辑，使用 `nHidden` 列表指定隐藏层维数。
 - 如果仍然使用单一参数 `size_list` 指定维数，那么 `size_list` 就需要是一个嵌套的列表结构。
 - 预期用户以 `[input_dimention, [hidden1, hidden2, ...], output_dimention]` 的方式给出参数。
 - 看上去有些复杂，所以不用这个思路。
 - 修改 `__init__()` 的参数列表，把原本的 `size_list` 拆分成 `input_dim`, `nHidden`, `output_dim` 三个参数。
 - 其中 `input_dim` 和 `output_dim` 是标量，`nHidden` 是列表。
 - 使用 `input_dim`, `nHidden`, `output_dim` 重构原本的 `size_list` 变量：

- 列表 [input_dim, *nHidden, output_dim] 即为原本的 size_list 变量。
- 后续代码可以沿用原 __init__()。

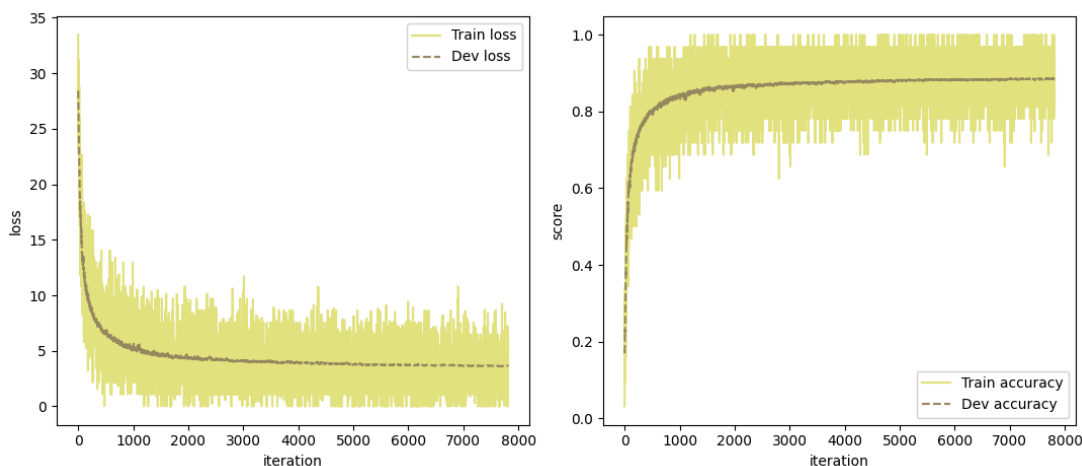
现在创建模型的方式变成了：

```
nn.models.Model_MLP(input_dim, nHidden, output_dim, act_func,
lambda_list)
```

修改网络架构，隐藏层维数 nHidden = [512, 128]，即网络具备两个隐藏层。
原本的 init_lr 似乎有点大，修改为 0.001，其余设置不变。

```
linear_model = nn.models.Model_MLP(train_imgs.shape[-1], [215, 128],
10, 'ReLU', [1e-4, 1e-4, 1e-4])
optimizer = nn.optimizer.SGD(init_lr=0.001, model=linear_model)
scheduler = nn.lr_scheduler.StepLR(optimizer=optimizer, step_size=400,
gamma=0.9)
loss_fn = nn.op.MultiCrossEntropyLoss(model=linear_model,
max_classes=train_labs.max()+1)
runner = nn.runner.RunnerM(linear_model, optimizer, nn.metric.accuracy,
loss_fn, scheduler=scheduler)
runner.train([train_imgs, train_labs], [valid_imgs, valid_labs],
num_epochs=5, log_iters=100, save_dir=r'./best_models')
```

训练结果：



训练集和验证集准确率约为 0.87，比前面用 [600] 作为隐藏层维数的准确率低，后续仍然沿用 [600] 作为隐藏层维数。

2. 修改 lr_scheduler.py，实现不同学习率策略。

2.1 lr_scheduler.py: MultiStepLR

```
MultiStepLR(optimizer, milestones, gamma)
```

在由 milestones 定义的多个训练阶段中，按照固定比例 gamma 衰减学习率。

- 初始化 __init__()
 - 按照常规操作初始化（调用父类构造函数，参数存入私有变量中）。
- 单步调整策略 step()

- 参照已实现的 StepLR，但是条件语句变为判断当前步数 `self.step_count` 是否在给定的 `self.milestones` 中。
在更新学习率后无需重置 `self.step_count`，`self.step_count` 在训练过程中一直累加。

2.2 lr_scheduler.py: ExponentialLR

`ExponentialLR(optimizer, gamma)`

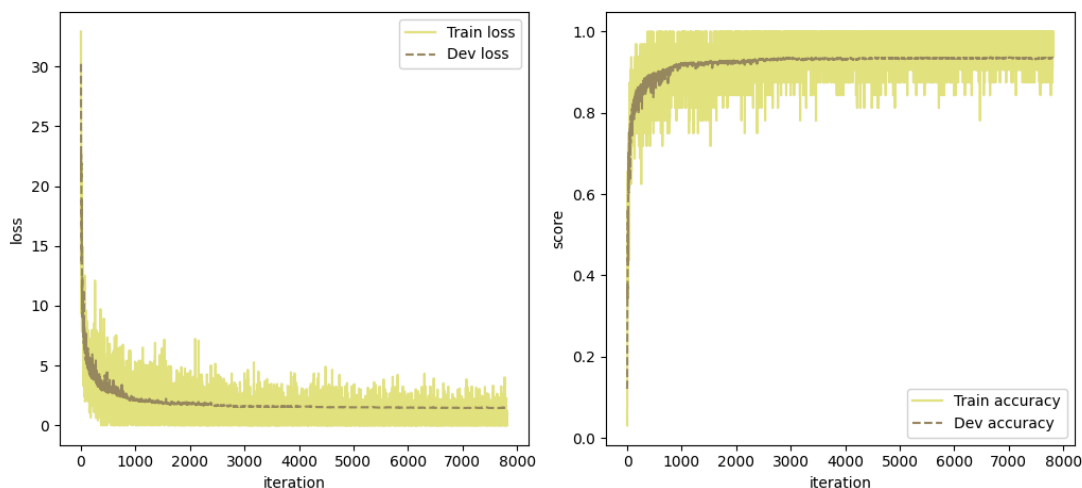
每步的学习率会乘以固定的衰减因子 `gamma`

- 初始化 `__init__`
 - 同样按照常规操作初始化。
- 单步调整策略 `step()`
 - 无需条件判断，每一步都更新学习率。

用原本的 `test_train.py` 运行（用 MultiStepLR）：

```
linear_model = nn.models.Model_MLP(train_imgs.shape[-1], [600], 10,
'ReLU', [1e-4, 1e-4])
optimizer = nn.optimizer.SGD(init_lr=0.06, model=linear_model)
scheduler = nn.lr_scheduler.MultiStepLR(optimizer=optimizer,
milestones=[800, 2400, 4000], gamma=0.5)
loss_fn = nn.op.MultiCrossEntropyLoss(model=linear_model,
max_classes=train_labs.max()+1)
runner = nn.runner.RunnerM(linear_model, optimizer, nn.metric.accuracy,
loss_fn, scheduler=scheduler)
runner.train([train_imgs, train_labs], [valid_imgs, valid_labs],
num_epochs=5, log_iters=100, save_dir=r'./best_models')
```

训练结果：



训练集和验证集上的准确率约为 0.94。

3. 尝试 L2 正则化

3.1 optimizer.py: SGD 中已实现 L2 正则化

在 optimizer.py 中已经实现了 L2 正则化的参数更新。

SGD.step() 中，在由梯度更新参数前，会先根据 layer.weight_decay 和 layer.weight_decay_lambda 的值更新参数。

- 具体而言，如果 layer.weight_decay 设为 True，则将参数都变为原本的 $(1-\lambda)$ 倍。
- 等价于损失函数中添加系数为 $\text{layer.weight_decay_lambda}/2$ 的 L2 正则化损失后的反向传播。

3.2 op.py: L2Regularization

应该不需要额外在 op.py 的 L2Regularization 类中实现参数更新。

原本的正则化相关实现：

- 正则化参数 λ 在模型初始化时，通过 Model_MLP 的 lambda_list 参数设定。
- Model_MLP 在初始化线性层时，会把 $\text{lambda_list}[i]$ 存入线性层的 $\text{layer.weight_decay_lambda}$ 中，并设置 $\text{layer.weight_decay} = \text{True}$ 。
- 优化器 optimizer.py 中，在用梯度更新层的参数前，会先根据 $\text{layer.weight_decay}$ 和 $\text{layer.weight_decay_lambda}$ 的值对参数添加正则化效果。

那么 op.py 的 L2Regularization 类的意义或许是作为添加了正则化项后的“损失函数” loss_fn 。

- 但是考虑到 test_train.py 中损失函数的定义方式，如果直接用 L2Regularization 类定义损失函数 loss_fn ，则需要在定义时额外提供诸如 MultiCrossEntropyLoss 类的实例作为参数，不太合理。
- L2 正则化项本身不参与链式的反向传播 backward()，将 L2Regularization 类作为 Layer 类的子类似乎也不太合理。
- 注意到为了实现反向传播，在 MultiCrossEntropyLoss 等表示损失的类中，backward() 方法末尾应该都会调用模型 model 的反向传播方法 $\text{self.model.backward}(\text{self.grads})$ 。也就是说，所有表示损失的类都需要提供模型 model 作为参数。
- 另一方面，基于先前的分析，L2Regularization 类的目的是计算 L2 正则化项。而 L2 正则化项仅和模型 model 本身有关。只需要提供模型实例 model，就可以根据 model.layers 中已设置的正则化超参 $\text{layer.weight_decay}$ 和 $\text{layer.weight_decay_lambda}$ ，以及 model.layers 中的参数 layer.params 完成 L2 正则化项的计算。

结合以上分析，只需要提供 model 实例即可计算其 L2 正则化项，而所有损失函数中必然需要提供 model 实例作为参数。

所以考虑把 L2 正则化项的计算合并到 MultiCrossEntropyLoss 等表示损失函数的类中。不再用“类”实现 L2Regularization，改成辅助函数 $\text{L2_regularization}()$ 。

- 辅助函数 $\text{L2_regularization}()$ 以模型类的实例 model 为参数，参考 SGD 的实现。
- 初始化返回的变量 res 为 0。
- 遍历 model.layers 中的每一层，如果 layer.optimizable 为 True 则继续。
- 参考 SGD，用 $\text{layer.params.keys}()$ 获取字典的键，遍历参数字典中的每一项。用 $\text{np.sum}(x^2)$ 计算 L2 正则化项，乘以正则化系数 $\text{layer.weight_decay_lambda}$

, 累加到结果 res 中。

实际上发现, 把 L2 损失项合并到损失函数后, 输出的损失值反而比较让人困惑, 所以最终**没有**把 L2 正则化项加到损失函数中。

但是修改了 runner.py, 输出日志时也一并输出正则化项。

4. 实现 CNN

4.1 op.py: conv2D

```
conv2D(in_channels, out_channels, kernel_size, stride, padding,
initialize_method, weight_decay, weight_decay_lambda)
```

4.1.1 conv2D: 原始思路, 用四层嵌套循环, 未向量化, 效率低

初始化 `__init__()`

- 调用父类的初始化方法, 部分参数存入类的私有变量中。
- 仿照线性层 Linear 的初始化方法, 初始化 self.W, self.b, self.grads, self.input, self.params。
 - self.W 维数 $\text{out_channels} * \text{in_channels} * \text{kernel_size} * \text{kernel_size}$, 存储所有的 out_channels 个卷积核。
 - self.b 维数 out_channels。一个卷积核的内积结果 (即结果 $\text{out_channels} * \text{new_H} * \text{new_W}$ 中的每一个 $\text{new_H} * \text{new_W}$) 对应一个偏置标量, 标量会被广播成 $\text{new_H} * \text{new_W}$ 和内积结果相加。

前向传播 forward()

- 仍然先从 self.params 字典中取出更新后的 W, b。
- 根据输入的 X 的大小获取 batch_size, H, W (输入 X 的长宽), 把输入 X 存入 self.input 供反向传播使用。
- 初始化输出:
 - 计算输出的长宽 new_H, new_W。输出的维数应该为 $[\text{batch_size}, \text{out_channels}, \text{new_H}, \text{new_W}]$ 。
 - 根据维数初始化输出为全零数组 (后续通过嵌套循环向其中填入具体数值)。
- 给 X 添加 padding, 使用嵌套循环逐元素填充输出。
 - 按照 batch, out_channel, 行, 列 的顺序嵌套循环。
 - 每次循环, 从输入中提取出需要和卷积核做内积的部分, 维数 $\text{in_channels} * k * k$ 。
 - 和 self.W 中对应的卷积核做内积, 结果加上偏置 self.b 的对应标量。self.W 和 self.b 中具体使用的部分由 out_channel 指定。

反向传播 backward()

- 根据输入的 X 的大小获取 batch_size, new_H, new_W (grad 的长宽, 即输出数组的长宽)。
- 给 X 添加 padding, 初始化 W 和 X_padded 的梯度, 形状分别和 W 和 X_padded 相同。
- 直接求出 b 的梯度 db: 对 grads 中除了 out_channels 的所有维度求和。

- 求 W 和 X_{padded} 的梯度 dW, dX_{padded} :
 - 使用嵌套循环，从每一个输出元素的角度出发，累加得到 W 和 X_{padded} 的梯度。
 - 仍然按照 $\text{batch}, \text{out_channel}, \text{行}, \text{列}$ 的顺序嵌套循环。
 - 输出中的每一个元素关联一个 $\text{in_channel} * k * k$ 的卷积核，以及输入 X_{padded} 中一块 $\text{in_channel} * k * k$ 的子区域。每次循环即更新这两个区域涉及元素的梯度。
 - 具体而言：在输入子区域的梯度上累加 卷积核 $W * \text{grads}$ 中对应的上游梯度值。在卷积核 W 的梯度上累加 输入子区域 $* \text{grads}$ 中对应的上游梯度值。
- 去掉 dX_{padding} 中多余的部分，得到 X 的梯度 dX 。
- 把 dW 和 db 存入 self.grads 字典中，返回 dX 。

4.1.2 conv2D: 向量化后的版本，效率较高

借助 LLM 将上面的 conv2D 向量化了。

初始化 `__init__()`

- 调用父类的初始化方法，部分参数存入类的私有变量中。
- 初始化卷积核权重 self.W 和偏置 self.b ，默认使用适配 ReLU 激活函数的 Kaiming 初始化（后面会详细介绍实现细节）：
 - self.W 的大小 $[\text{out_channels}, \text{in_channels}, \text{kernel_size}, \text{kernel_size}]$ 。
 - self.b 的大小 $[\text{out_channels}]$ 。
 - 尝试过用 `np.random.normal` 初始化，但是出现了损失下降、准确率和随机猜测相近的情况，可能是陷入局部最小值了。

前向传播 `forward()`

核心思路是将需要做“内积的部分”转换成向量(数组中的一个维数)

- 仍然先从 self.params 字典中取出更新后的 W, b 。
- 通过 `im2col()` 辅助函数将输入 X 的展开为 cols ，大小为 $[B, \text{in_channels} * \text{kernel_size}^2, H_{\text{out}} * W_{\text{out}}]$ 。
 - `im2col()` 将卷积核对应的每一个输入中的区域都展开成列，放入 cols 中。
 - cols 的每一列对应输入中一个 $\text{in_channels} * \text{kernel_size} * \text{kernel_size}$ 的局部区域。
该局部区域和某个卷积核做内积后，对应 $H_{\text{out}} * W_{\text{out}}$ 个输出中的其中一个。
- 计算前向传播的数值结果
 - 将卷积核 W 的维数转换为 $[\text{out_channels}, \text{in_channels} * \text{kernel_size} * \text{kernel_size}]$ 。
 - 计算 $W @ \text{cols} + b$ ，得到结果维数 $[B, \text{out_channels}, H_{\text{out}} * W_{\text{out}}]$ ，存放前向传播的数值结果。
- 将 output 转换维数为 $[B, \text{out_channels}, H_{\text{out}}, W_{\text{out}}]$ 。
 - 需要先算出 H_{out} 和 W_{out} 的具体值。
- 将输入 X 和展开后的 cols 存入 self.cache ，供反向传播使用。

反向传播 `backward()`

- 梯度初始化：从缓存中取出 cols 和输入 X。
- 计算参数梯度 dW, db:
 - dW: 通过 `np.einsum('bom,bim->oi', grads_flat, cols)` 计算。
 - 其中 `grads_flat` 是 `grads` 展平后的结果，形状 `[B, out_channels, H_out*W_out]`。
 - `cols` 维数 `[B, in_channels*kernel_size*kernel_size, H_out*W_out]`。
 - 相当于固定 `grads_flat` 和 `cols` 的第二维 (对卷积核`[out_channels, in_channels, kernel_size, kernel_size]`求梯度)，用 `col` 的最后一维 (卷积核的每一个元素对应的输入元素) 和梯度 `grads_flat` 做内积，再对每一个 batch 求和。
 - db: 对 `grads ([batch, out_channels, H_out, W_out])` 中除了 `out_channels` 的所有维度求和。
- 计算输入梯度 dX:
 - 将 `W` 的维数转换为 `[out_channels, in_channels*kernel_size\2]`。
 - 计算 `W.T @ grads_flat`，结果存入 `dcols`。
 - `dcols` 的形状为 `[in_channels*kernel_size*kernel_size, B*H_out*W_out]`。
 - `dcols` 中的每一个元素均存储 某一个卷积核元素 和 某一个梯度值 的乘积。
 - 通过 `col2im()` 辅助函数将 `dcols` 中的梯度值求和，返回输入 `X` 的梯度 `dX`。该函数中会处理 padding。
- 将 `dW` 和 `db` 存入 `self.grads`，返回 `dX`。

辅助函数 `im2col()`

定义于 `forward()` 内部，仅被 `forward()` 使用。

将每一个卷积核 `W` 在输入 `X` 中对应的每一个区域都展开成列(长度 `C*kernel_size*kernel_size`)，

返回 `cols: [B, (C*kernel_size*kernel_size), (H_out*W_out)]`，便于卷积操作的矩阵化计算。

- 对输入 `X` 进行 padding，填充后的形状为 `[B, C, H+2*padding, W+2*padding]`。
- 使用 `sliding_window_view()` 提取所有滑动窗口到 `windows` 变量中。
 - `windows` 变量的维数 `[B, H_out, W_out, C, kernel_size, kernel_size]`。
 - 表示所有的 `B*H_out*W_out` 个输出值对应的输入 `X` 的区域。
 - 一个区域的维数为 `[C, kernel_size, kernel_size]`，和一个卷积核对应。
 - 卷积核和这些区域分别做内积操作即可得到输出值。
- 将 `windows` 变量转换维数为 `[B, C*kernel_size*C*kernel_size, H_out*W_out]`。

输入 `X` 中的单个元素可能会被重复存储很多遍(做一次卷积就存一遍)，空间复杂度变大，但是后续可以使用向量化运算代替嵌套循环，时间复杂度减少。

辅助函数 `col2im()`

定义于 `backward()` 内部，仅被 `backward()` 使用。

将存储输入 `X` 梯度的各个组成部分的 `dcols` (维数 `[B, C*kernel_size*kernel_size, H_out*W_out]`) 转换为输入 `X` 的梯度 `dX`。

- 根据输入 X 和 padding 等计算 X_padded 的维数，初始化填充后的梯度矩阵 dX_padded。
- 使用两层嵌套循环，逐窗口累加梯度：
 - 因为 im2col() 中 sliding_window_view() 返回的窗口是只读的，而且两次嵌套循环的实际运行速度可以接受，所以不尝试向量化了。
 - 遍历所有可能的窗口位置 (h, w)，计算对应 X_padded 的起始/结束位置。
 - 提取 dcols 中对应窗口的值 patch，形状为 [B, C, kernel_size, kernel_size]。
 - 将 patch 累加到 X_padded 的对应区域（由之前计算的起始/结束位置指定）。
- 去除 padding，返回梯度 dX。

4.2 op.py: MaxPooling

MaxPooling(kernel_size, stride, padding)

4.2.1 MaxPooling：原始思路，和 conv2D 类似，仍然使用四层嵌套循环。

- 和 conv2D 的原始思路类似；且没有考虑到多个 pooling 区域重叠的情况（步长 stride 大于单次池化区域边长 kernel_size）。
- 这个实现效率低且存在一定逻辑问题，故不详细叙述。

4.2.2 MaxPooling：向量化后错误的实现，但在大数据集上的效果好。

- 此时我还没有发现 pooling 区域可能重叠的问题，只是对上面的实现做了向量化。
- 但是 pooling 区域并不是这里的主要矛盾，毕竟我写的 test_train_CNN.py 中设置了 stride = 2, kernel_size = 2，在实际运行时不会出现 pooling 区域重叠的情况，按理说应该和正确的实现是等价的。
- 关键的错误在于反向传播中，根据掩码 mask 传递梯度的部分：
`dX_padded[self.mask] = grads.reshape(-1)`
 - 其中，mask 用来指示 X_padded (输入的 X 加 padding 后的结果) 中被 pooling 过程“选中”的最大值位置。
 - 上面的代码看上去似乎很正确，根据 mask 中存储的布尔值选出具有非零梯度的位置，再将上游梯度 grads 展平赋值给这些位置。
 - 但是 mask 和 X_padded 的尺寸相同，相比 grads 的尺寸比较大的，grads 中的一行会对应 mask 中的多行。
 - 这就导致 dX_padded[self.mask] 按行展开后的列表和 grads.reshape(-1) 同样按行展开后的列表存在“错位”。

举个例子：

```
# X_padded 大小 4*4
# kernel_size = 2, stride = 2

# X_padded 的取值：
X_padded = [[1, 2, 3, 4],
             [3, 4, 1, 2],
             [2, 1, 4, 3],
             [4, 3, 2, 1]]
```

```

# 则 mask 的取值为:
mask = [[0, 0, 0, 1],
        [0, 1, 0, 0],
        [0, 0, 1, 0],
        [1, 0, 0, 0]]

# 上游梯度 grads 的取值:
grads = [[5, 6],
          [7, 8]]

# 我们期望反向传播后 dX_padded 的取值:
expected_dX_padded = [[0, 0, 0, 6],
                      [0, 5, 0, 0],
                      [0, 0, 8, 0],
                      [7, 0, 0, 0]]

# 但是实际上 dX_padded[self.mask] = grads.reshape(-1) 的逻辑:
'''
dX_padded[mask] 取出的元素:
    [ dX_padded[0,3], dX_padded[1,1], dX_padded[2,2], dX_padded[3,0] ]
grads.reshape(-1) 的结果: [5, 6, 7, 8]
'''

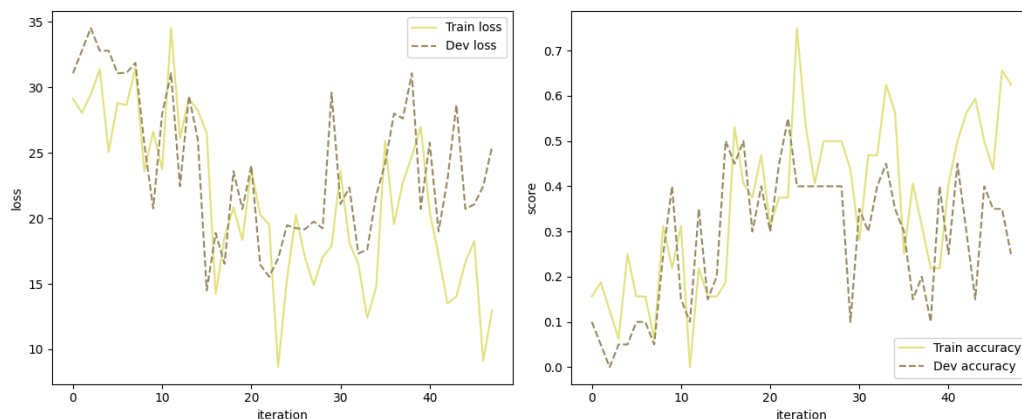
# 对应赋值后, 实际得到的 dX_padded:
reality_dX_padded = [[0, 0, 0, 5],
                     [0, 6, 0, 0],
                     [0, 0, 7, 0],
                     [8, 0, 0, 0]]

# 可以看到梯度错位了

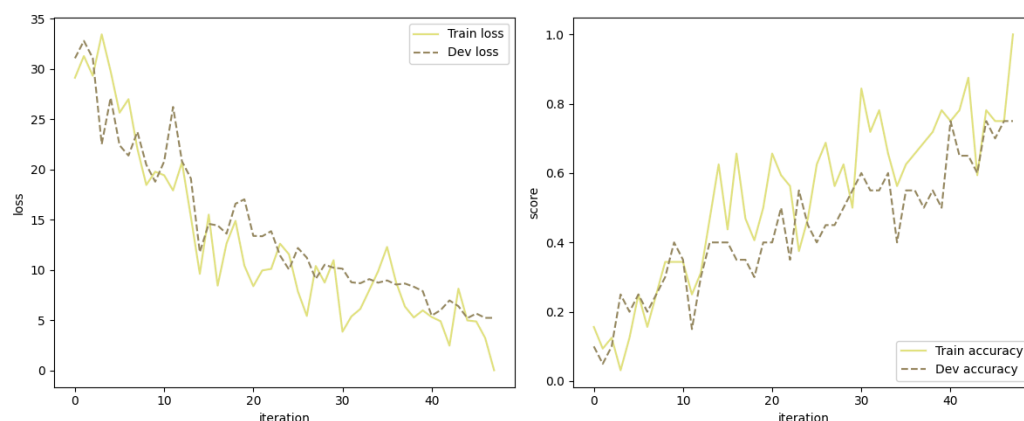
```

- 因为仅仅是“梯度错位”而已，所以代码也能跑，甚至可以做到在小数据集和大数据集上让损失下降并且让准确率提升。
- 一开始只是发现：这一个“没有考虑 pooling 范围重叠”的实现和后续修改的“考虑了 pooling 范围重叠”的实现，在理论上等价的情况下（kernel_size = stride = 2），在小数据集上拟合的效果不同。
具体而言，这种“错误”实现在小数据集上的损失下降更慢，过拟合速度也更慢。这是意料之中的。

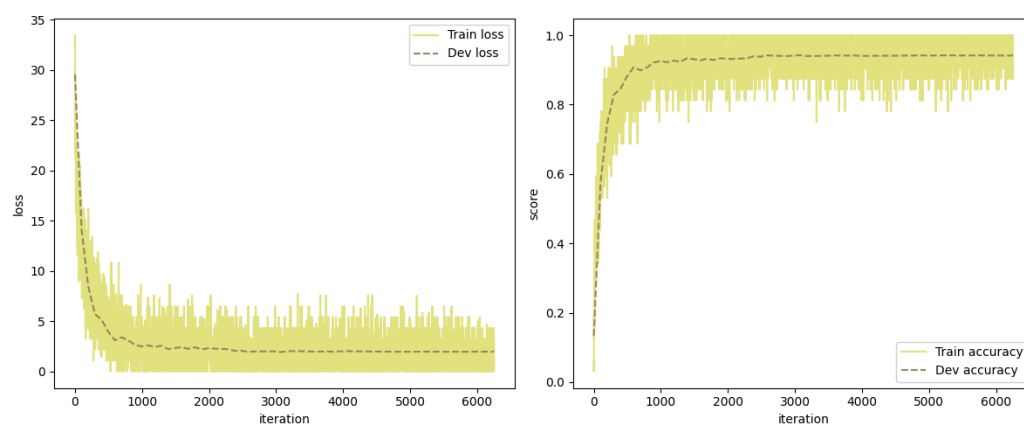
错误代码在小数据集上的曲线（360 条训练集，20 条验证集，训练 4 epoch）：



正确实现的代码在小数据集上的曲线（相同的 360 条训练集，20 条验证集，训练 4 epoch）：

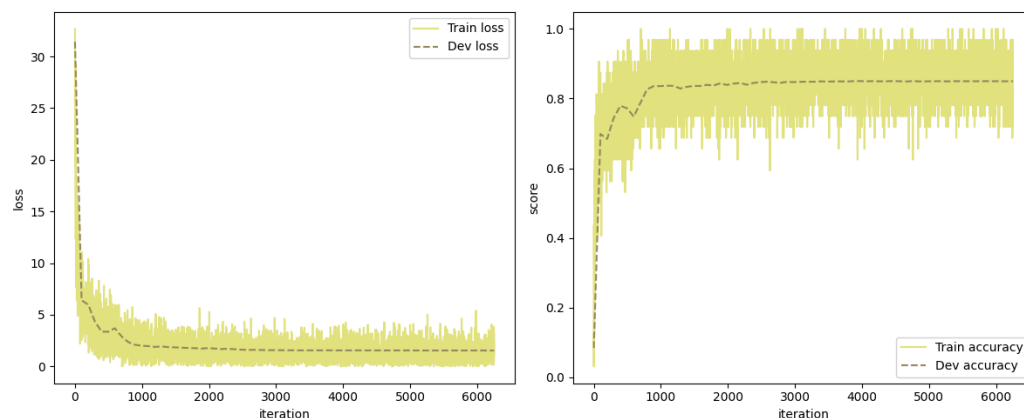


- 但是这种“错误”实现在大数据集上的效果更好。虽然损失下降较慢，但准确率提升更快，最终达到的准确率也更高。错误代码在大数据集上的曲线（50000 条训练集，10000 条验证集，除了 log_iters 以外的其余配置和小数据集相同，训练 4 epoch）：



在验证集和测试集上的准确率约为 0.94。

正确实现的代码在大数据集上的曲线（50000 条训练集，10000 条验证集，训练 4 epoch）：



在验证集和测试集上的准确率约为 0.84。

可能是因为这种“梯度错位”相当于引入了一些不算大的噪声：

- 一方面，适当的噪声使得参数可以跳出局部最小值，让模型获得更强的“泛化能力”，准确率更高。

由于噪声和“泛化能力”相关，故在小数据集上过拟合时，“梯度错位”的效果会不如梯度下降的正确实现的效果。

- 另一方面，噪声使得参数并不严格沿负梯度方向更新，损失下降缓慢。
- 总之，这个错误引起的“梯度错位”，或许可以视为是对参数的更新方向进行了可以接受的扰动。
 - 其效果应该可以类比 Momentum 等优化算法，不同之处在于，这个错误的实现对梯度的扰动方向是较为随机的。
 - 如果要在修正了“pooling 范围重叠”问题的代码上添加这个效果，或许可以在反向传播中增加一些代码修改返回的梯度。
 - 后续尝试了 Moment GD，其性能确实比“梯度错位”得到的结果更好。

4.2.3 MaxPooling: 正确的向量化实现

考虑了 pooling 区域重叠的问题，使用了维数为 [B, C, H_out, W_out] 的数组存储最大值索引。

初始化 `__init__()`

- 调用父类的初始化方法，部分参数存入类的私有变量中。
- 标记 `self.optimizable = False`。

前向传播 `forward()`

- 对输入 X 进行 padding，填充值为 `-np.inf`，避免影响后续取最大值。
- 提取窗口：
 - 使用 `sliding_window_view()` 提取滑动窗口 windows。
 - 维数 [B, C, H_out, W_out, kernel_size, kernel_size]，参照之前 conv2D 向量化的解释。
- 取窗口中的最大值，记录最大值在窗口内的索引：
 - 展平 windows 为向量，形状 [B, C, H_out, W_out, kernel_size*kernel_size]，为了后续取 `argmax`。
 - 对每个 `kernel_size*kernel_size` 窗口作用 `argmax`，得到最大值位置索引 `max_indices`，`max_indices` 维数 [B, C, H_out, W_out]。
 - 根据 `max_indices` 和 `self.kernel_size` 得到最大值对应的行号 `rows` 和列号 `cols`，维数也为 [B, C, H_out, W_out]。
 - 这里需要用维数为 [B, C, H_out, W_out] 的数组存储最大值索引，不能直接用一个大和 X_padded 相同的掩码，为了照顾到 pooling 区域重叠的情况。
- 把上述窗口内部的索引转化为 X_padded 中的索引：
 - 计算窗口起始位置 `h_start` 和 `w_start`。
 - 起始位置和窗口内的偏移量相加，计算填充后的绝对坐标 `max_h` 和 `max_w`。
 - 将 `max_h` 和 `max_w` 存入私有变量中，供反向传播使用。
- 返回最大值组成的数组 `flat_windows.max(axis=-1)`。

反向传播 `backward()`

- 初始化全零数组 `dX_padded`，大小和 `grads` 一致。
- 利用缓存的 `max_h` 和 `max_w`，通过 `np.add.at` 将梯度 `grads` 累加到对应位置。
- 从 `dX_padded` 中移除 padding，返回 `dX`。

在相同的配置下，向量化后的版本在较少样本上训练一个 epoch 的时间是 3.92s，未量化的版本则需要至少 400s。

4.3 op.py: Flatten

`Flatten()`

用于最后一个 conv2D 卷积层之后，把卷积层的结果展开成一维数组，对接后续的全连接层。

初始化 `__init__()`

- 调用父类的初始化方法，设置 `self.optimizable` 为 `False`。

- 初始化 `self.input_shape`，用于反向传播中重塑上游梯度维数。

前向传播 `forward()`

- 将输入 `X` 的原始形状保存到 `self.input_shape` 中。
- 重塑输入 `X` 的维数：保留第一维 `batch_size`，其余维数展平。

反向传播 `backward()`

- 由于前向传播不涉及数值运算，这里直接将上游梯度 `grads` 重塑为和输入 `X` 相同的维数即可（前面已经存下了 `self.input_shape`）。

4.4 models.py: 卷积神经网络 Model_CNN 类

`Model_CNN(layers)`

较 `Model_MLP` 的改动比较大。

后续如果在 `op.py` 中增加其它层的实现，需要在 `models.py` 的 `Model_CNN` 的 `load_model` 和 `save_model` 中增加对应存取逻辑。

初始化 `__init__()`

- 考虑到 CNN 的结构一般比较复杂，可能含有 卷积层、池化层、激活函数、全连接层等多种层，原本 `Model_MLP` 的初始化方式很难适用于 CNN。
- 需要在创建 `Model_CNN` 类的实例时，提供包含所有层的实例的列表 `layers`。
- 初始化方法中直接将提供的列表 `layers` 存入私有变量 `self.layers` 中。

前向传播 `forward()`

- 参照 `Model_MLP` 中的 `forward()`。
- 检查模型是否初始化的语句的实现，修改为检查 `self.layers` 是否为空。

反向传播 `backward()`

- 参照 `Model_MLP` 中的 `backward()`。

存储模型 `save_model()`

`Model_MLP` 的 `save_model()` 将每一个线性层的输入/输出维数存入 `param_list[0]`，将激活函数存入 `param_list[1]`，`param_list` 的后续元素则被用来存储线性层的参数 `W` 和 `b`。

由于 CNN 的复杂性，同样需要修改这里的逻辑。

- `save_model()` 的核心在于把模型各个层的信息格式化后写入文件，以保证 `load_model()` 中可以根据文件中的信息重构模型。
- 仍然以“层”的视角，列表 `param_list` 的每一个元素是一个字典，存储一层的信息。
- 一层的信息包含：层的类型、创建层所需要的超参数（和每层的初始化方法对应）、层中的可优化参数（若层本身可优化）。
- 使用 `isinstance()` 结合条件判断语句，对每一种类型的层都单独处理：将当前层的上述信息存入字典中，作为列表 `param_list` 的新元素。
层的类型存储在 `'type'` 字段中。
- 如果层的类型不属于任何一种在条件判断语句中给出的类型，会抛出错误信息。
- 最后参考 `Model_MLP`，将列表 `param_list` 写入文件。

加载模型 load_model()

- 参考 Model_MLP，将文件内容读取到列表 param_list 中。
- 根据 save_model() 的逻辑，遍历 param_list 的元素，逐层重构。
- 从每一个元素的 'type' 字段中获取层的类型，结合条件判断语句，对每一个类型的层单独处理：
根据存储的超参重新初始化各个层。
若当前层有可优化参数，需要额外设置层的实例中的 W 和 b。
- 如果层的类型不属于任何一种在条件判断语句中给出的类型，会抛出错误信息。
- 最后将层的实例组成的列表 layers 存入 self.layers。

4.5 其它适配

4.5.1 runner.py

原本的逻辑是每一个 batch 在开发集上测一遍准确率，但是 CNN 前向传播比较慢，每个 batch 上都跑一遍验证集会很慢。

- 如果是 MLP 模型（通过 self.model 判断），则仍然每个 batch 都在开发集上测试。
- 如果是 CNN 模型，则只在需要输出日志 ((iteration) % log_iters == 0) 的时候在开发集上测试。
- 涉及 runner.py 的 line 65 - line 75

4.5.2 plot.py

由于 runner.py 中修改了在开发集上测试的时机，plot.py 中绘制训练集和开发集损失曲线的部分也要一并修改。

- 新增 plot_cnn()，在原有基础上修改。
- 使用 np.linspace() 为 dev_loss 和 dev_scores 生成开发集的 x 轴坐标。
- 绘图时提供相应的 x 轴坐标即可。

4.5.3 test_train_CNN.py 和 test_model_CNN.py

CNN 和 MLP 的输入不同，MLP 要求输入是展平的一维数组，CNN 则要求输入是 C*H*W 的三维数组。

- 在原本的训练/测试/验证数据基础上增加 .reshape() 操作。
- MNIST 图片均为灰度图像，只有一个 channel，每一张图片大小为 28*28。
- 训练集上的 .reshape() 例子：
 - 获取训练集长度: N_train = train_imgs.shape[0]
 - 转换维数: train_imgs = train_imgs.reshape(N_train, 1, 28, 28)
- 最前面的 np.frombuffer(f.read(), dtype=np.uint8) 直接读入的内容似乎是一个很长的
一维数组，还是在给定的代码后面增加 .reshape() 比较方便。

得到上述四张 CNN 图像的超参设置：

均使用默认的初始化方式: conv2D 层用 Kaiming 初始化，Linear 层用标准正态初始

化。

详细配置：

```
import mynn as nn

# ...

MODE = 'debug'
# MODE = 'train'

if MODE == 'debug':
    valid_imgs = train_imgs[:20] # 验证集
    valid_labs = train_labs[:20]
    train_imgs = train_imgs[20:380] # 训练集
    train_labs = train_labs[20:380]
    logiters = 1
    numepochs = 4
elif MODE == 'train':
    logiters = 100
    numepochs = 4

# 初始化模型
cnn_model = nn.models.Model_CNN(
    layers = [
        # bs*1*28*28 --> bs*32*28*28
        nn.op.conv2D(in_channels=1, out_channels=32, kernel_size=3,
padding=1),
        nn.op.ReLU(),
        # bs*32*28*28 --> bs*32*14*14
        nn.op.MaxPooling(2, 2),

        # bs*32*14*14 --> bs*64*14*14
        nn.op.conv2D(in_channels=32, out_channels=64, kernel_size=3,
padding=1),
        nn.op.ReLU(),
        # bs*64*14*14 --> bs*64*7*7
        nn.op.MaxPooling(2, 2),

        # 两个全连接层
        nn.op.Flatten(),
        nn.op.Linear(64*7*7, 512),
        nn.op.ReLU(),
        nn.op.Linear(512, 10)
    ])

optimizer = nn.optimizer.SGD(init_lr=1e-3, model=cnn_model)

scheduler = nn.lr_scheduler.MultiStepLR(optimizer=optimizer,
milestones=[800, 2400, 3200, 4800], gamma=0.2)

loss_fn = nn.op.MultiCrossEntropyLoss(model=cnn_model, max_classes=10)

runner = nn.runner.RunnerM(cnn_model, optimizer, nn.metric.accuracy,
loss_fn, scheduler=)
```

```
runner.train([train_imgs, train_labs],
             [valid_imgs, valid_labs],
             num_epochs=numepochs,
             log_iters=logiters,
             save_dir=r'./cnn_models')
```

5. Moment GD 优化器

5.1 Moment GD 的实现

5.1.1 optimizer.py: MomentGD

```
MomentGD(init_lr, model, mu)
```

- 初始化 `__init__()`
 - 调用父类初始化方法。
 - 需要额外提供参数 `mu`，作为动量项更新时原本动量项之前的系数，存入 `self.mu` 中。
 - 初始化 `self.v` 为空列表，存放各层参数的动量项。
 - 列表中的每一项对应一个 `layer.params` 字典中的项。
- `step()`
 - 先初始化 `i = 0`，用于索引 `self.v` 列表中的项。
 - 参照 SGD 的 `step()` 方法，遍历各层、检查当前层是否具有可优化参数、获取 `layer.params` 字典中的键。
 - 先对当前层进行 `weight_decay`。
 - 更新动量，并根据动量更新参数：
 - 第一轮循环时 `self.v` 列表为空，需要一边循环一边初始化 `self.v`。
 - 更新动量前，先检查 `self.v` 列表的长度是否大于当前层对应的索引 `i`，若不大于则需要初始化 `self.v` 为零数组，维数和当前层的参数相同。
 - 按照公式，根据 `mu, lr, layer.grads[key]` 的值更新 `self.v[i]`。
 - 用 `self.v[i]` 更新参数 `layer.grads[key]`。

5.1.2 op.py: Kaiming 初始化

原本默认的参数初始化方式 (conv2D 用 Kaiming 初始化，Linear 层用标准正态初始化) 在使用 Moment GD 优化器时，会使 CNN 陷入局部最小值，故尝试修改 Linear 层的参数初始化方式，同样使用 Kaiming 初始化。

修改 `op.py` 中的参数初始化部分：

- 由于不同类型的层对应的 Kaiming 初始化方式不一样，考虑直接在层内实现 Kaiming 初始化。
 - 虽然目前只用线性层 Linear 和卷积层 conv2D 涉及到参数初始化，可以通过传入的 `size` 参数的数量确定层的类型。
 - 但考虑到代码的可扩展性，在两个层的初始化方法内部分别实现了 Kaiming 初始化。
- 若传入的 `initialize_method` 参数的值为 'Kaiming' 或 'He' 则使用 Kaiming 初始化，否则维持原有初始化逻辑。

- 线性层 Linear 的 Kaiming 初始化
 - 参数 W 使用标准正态除以 $\sqrt{2.0/\text{in_dim}}$ 初始化。
 - 参数 b 初始化为全 0。
- 卷积层 conv2D 的 Kaiming 初始化。
 - 参数 W 使用标准正态除以 $\sqrt{2.0/(\text{in_channels} * \text{kernel_size}^2)}$ 初始化。
 - 参数 b 初始化为全 0。

可以用字符串 'Kaiming' 或 'He' 指定使用 Kaiming 初始化作为参数初始化方式。

5.1.3 models.py: 支持创建 MLP 时指定参数初始化方式

原本 Model_MLP 在被创建时不支持自定义参数初始化方式。

修改 Model_MLP 类中的初始化方法 `__init__()`:

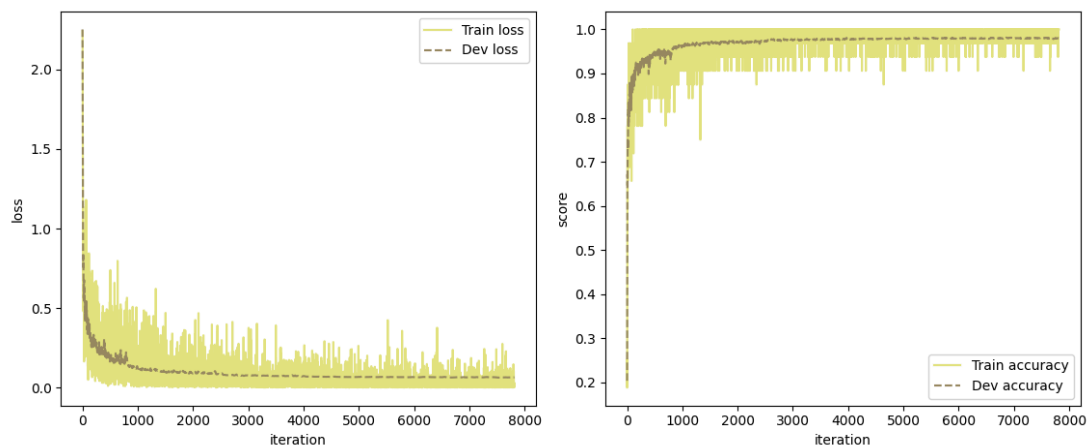
- 增加参数 `initialization_method`, 默认为 `np.random.normal`, 和 `op.py` 的 Linear 类一致。
- 修改创建全连接层的一行 (`layer = Linear(...)`), 指定 `initialization_method` 参数。

5.2 训练 MLP

5.2.1 MLP: Moment GD + Kaiming

使用 Moment GD 配合 Kaiming 初始化, 训练 MLP。

仍然用 50000 条训练集, 10000 条验证集, 训练 5 epoch:



验证集上准确率约为 0.9800, 测试集上准确率约为 0.9795。

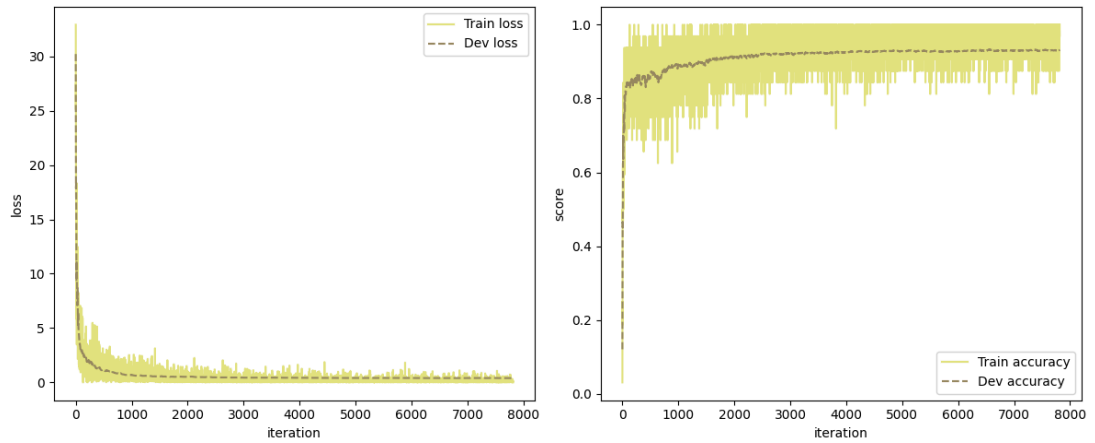
修改的配置:

```
# 定义模型, 使用 Kaiming 初始化
linear_model = nn.models.Model_MLP(train_imgs.shape[-1], [600], 10,
    'ReLU', [1e-4, 1e-4], initialization_method='Kaiming')
# 定义优化器, 改用 Moment GD, init_lr 维持不变, mu 使用默认值 0.9
optimizer = nn.optimizer.MomentGD(init_lr=0.06, model=linear_model,
    mu=0.9)
```

5.2.2 MLP: Moment GD + StdNormal

使用 Moment GD 配合标准正态初始化, 训练 MLP。

仍然用 50000 条训练集, 10000 条验证集, 训练 5 epoch:



验证集上准确率约为 0.9314，测试集上准确率约为 0.9278。

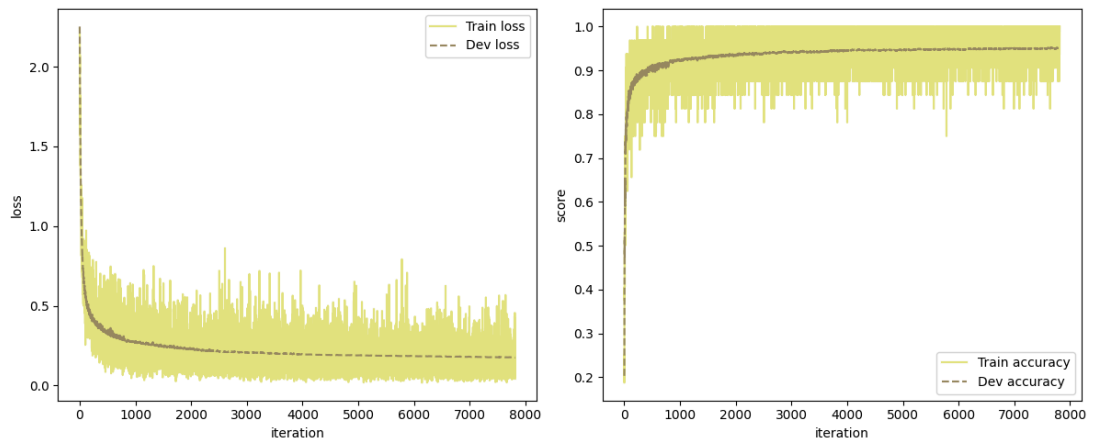
修改的配置：

```
# 定义模型，用默认的初始化方式
linear_model = nn.models.Model_MLP(train_imgs.shape[-1], [600], 10,
    'ReLU', [1e-4, 1e-4])
# 定义优化器，用 Moment GD
optimizer = nn.optimizer.MomentGD(init_lr=0.06, model=linear_model,
    mu=0.9)
```

5.2.3 MLP: SGD + Kaiming

使用 SGD 配合 Kaiming 初始化，训练 MLP。

仍然用 50000 条训练集，10000 条验证集，训练 5 epoch:



验证集上准确率约为 0.9511，测试集上准确率约为 0.9534。

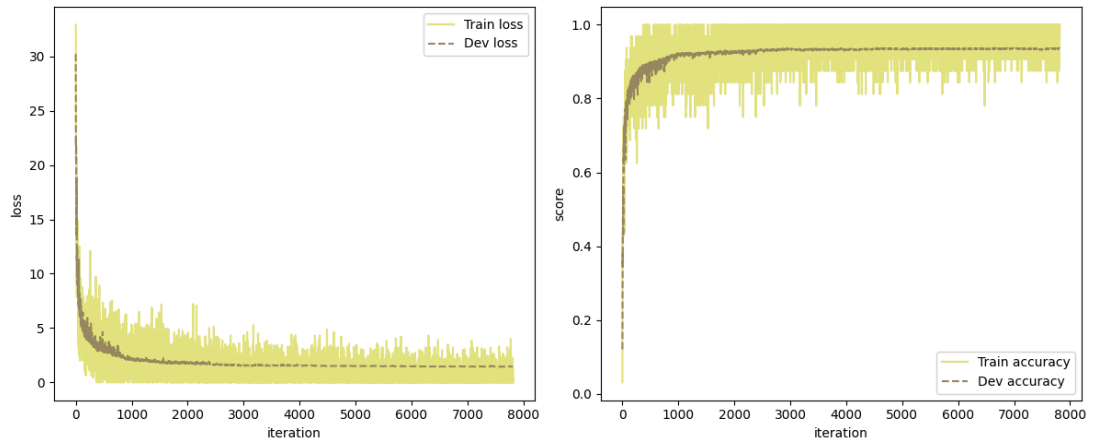
修改的配置：

```
# 定义模型，使用 Kaiming 初始化
linear_model = nn.models.Model_MLP(train_imgs.shape[-1], [600], 10,
    'ReLU', [1e-4, 1e-4], initialization_method='Kaiming')
# 定义优化器，用 SGD
optimizer = nn.optimizer.SGD(init_lr=0.06, model=linear_model)
```

5.2.4 MLP: SGD + StdNormal

使用 SGD 配合标准正态初始化，训练 MLP。

仍然用 50000 条训练集，10000 条验证集，训练 5 epoch:



验证集上准确率约为 0.9360，测试集上准确率约为 0.9360。

修改的配置：

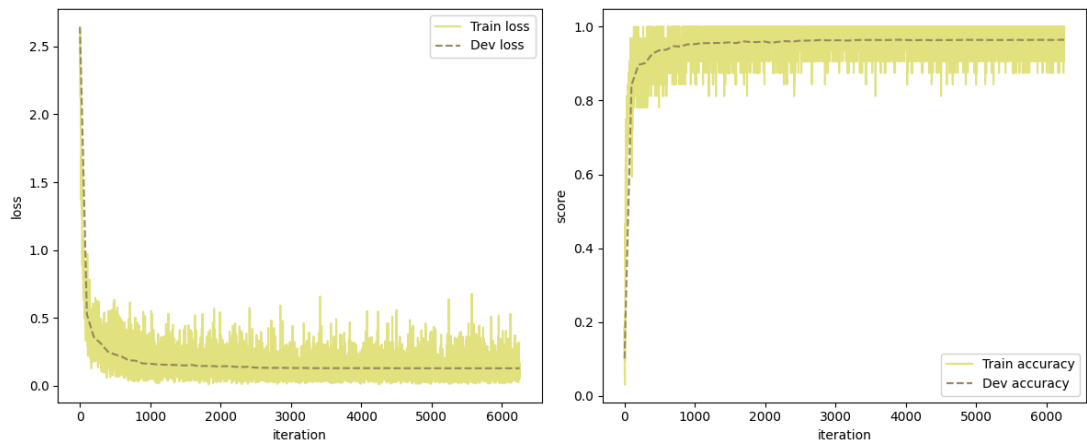
```
# 定义模型，用默认的初始化方式
linear_model = nn.models.Model_MLP(train_imgs.shape[-1], [600], 10,
    'ReLU', [1e-4, 1e-4])
# 定义优化器，用 Moment GD
optimizer = nn.optimizer.SGD(init_lr=0.06, model=linear_model)
```

5.3 训练 CNN

5.3.1 CNN: Moment GD + Kaiming + Kaiming

使用 Moment GD 配合 Kaiming 初始化 (conv2D 和 Linear 层均使用 Kaiming 初始化) 训练 CNN。

仍然用 50000 条训练集，10000 条验证集，训练 4 epoch:



验证集上准确率约为 0.9640，测试集上准确率约为 0.9639。

修改的配置：

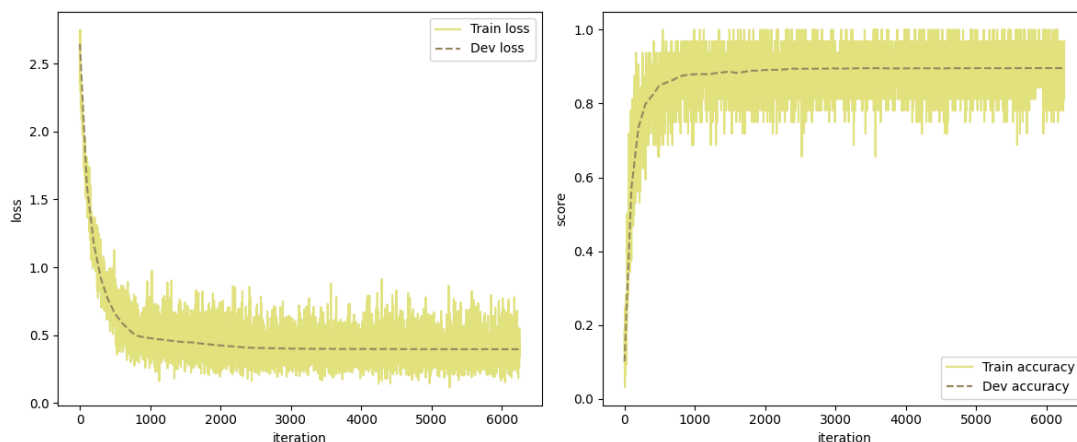
```
# 模型定义部分，规定线性层使用 Kaiming 初始化
nn.op.Linear(64*7*7, 512, initialize_method='Kaiming'),
nn.op.Linear(512, 10, initialize_method='Kaiming')

# 优化器定义部分，改用 Moment GD, init_lr 维持不变, mu 使用默认值 0.9
optimizer = nn.optimizer.MomentGD(init_lr=1e-3, model=cnn_model,
    mu=0.9)
```

5.3.2 CNN: SGD + Kaiming + Kaiming

补测使用 SGD 配合 Kaiming 初始化 (conv2D 和 Linear 层均使用 Kaiming 初始化) 训练 CNN。

仍然用 50000 条训练集，10000 条验证集，训练 4 epoch:



在验证集上的准确率约为 0.8962，测试集上的准确率约为 0.9007。

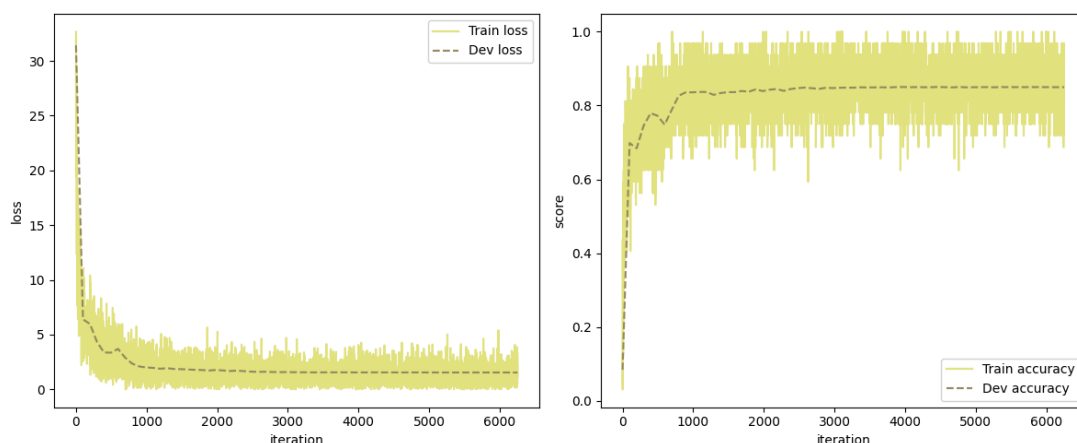
修改的配置：

```
# 模型定义部分，规定线性层使用 Kaiming 初始化
nn.op.Linear(64*7*7, 512, initialize_method='Kaiming'),
nn.op.Linear(512, 10, initialize_method='Kaiming')

# 优化器定义部分，改用 SGD, init_lr 维持不变
optimizer = nn.optimizer.SGD(init_lr=1e-3, model=cnn_model)
```

5.3.3 CN: SGD + Kaiming + StdNormal

之前使用 SGD 配合 conv2D 层的 Kaiming 初始化 (Linear 层使用标准正态初始化) 训练 CNN 得到的图像:



在验证集的准确率约为 0.8422，在测试集上的准确率约为 0.8420。

使用的配置：

```
# 模型定义部分，线性层使用默认的标准正态初始化
nn.op.Linear(64*7*7, 512),
nn.op.Linear(512, 10)
```

```
# 优化器定义部分, 改用 SGD, init_lr 维持不变
optimizer = nn.optimizer.SGD(init_lr=1e-3, model=cnn_model)
```

结论: 对于 MLP 和 CNN, Moment GD + 线性层和卷积层的 Kaiming 初始化 > SGD + 线性层和卷积层的 Kaiming 初始化 > SGD + 卷积层的 Kaiming 初始化和线性层的标准正态初始化。

6. 数据增强: data_augmentation.py

6.1 较为底层的实现

实现图像的平移/旋转/缩放。

新建 data_augmentation.py, 在其中实现了: 双线性插值 + 平移 + 旋转 + 缩放。

6.1.1 双线性插值

`BiLinearInterpolation(imgs, grid_h, grid_w, padding)`

成批地处理图像, 被平移/旋转/缩放函数调用。

- 参数:
 - imgs: 以 [B, C, H, W] 的形式给出的一批“原图”。
 - grid_h, grid_w: [B, C, H', W'] 数组, 指定返回图像中的每一个元素在原图像 imgs 中对应的像素位置。
H', W' 是返回图像的长/宽, 不一定要和原图的 H, W 相同。
 - padding: 当给出的 grid_h, grid_w 元素超界时, 返回的图像中对应像素的取值。
- 返回值:
 - 和 grid_h, grid_w 大小相同的数组, 通过双线性插值得到。
- 具体实现:
 - 从 grid_h, grid_w 中提取整数部分 h,w 和小数部分 dh,dw 。
 - 后续用 [h, w], [h, w+1], [h+1, w], [h+1, w+1] 作为原图 imgs 的索引, 取出双线性插值计算所需的原图中的元素。
 - 用 dh, dw 得到各个元素的权重。
 - 处理索引超界问题:
 - 把 h,w 中原本不超界且 +1 后不超界 (在 [0, H-1] 内) 的元素的掩码存入 valid 数组中, 结果中 valid 未覆盖的区域都需要用 padding 填充。
 - 对 h,w 中原本超界或 +1 后会超界的元素, 截断到 [0, H-2] 范围内。
 - 利用广播机制, 从 imgs 中取出 4 个 [B, C, H', W'] 的数组, 分别对应 [h, w], [h, w+1], [h+1, w], [h+1, w+1]。
各自乘以对应的权重, 累加。
 - 最后把非 valid 的地方填充 padding 。

6.1.2 图像的批量平移

`shift2D(imgs, shift_h, shift_w, padding)`

- 参数:
 - imgs: 以 [B, C, H, W] 的形式给出的一批“原图”。
 - shift_h, shift_w: 大小为 B 的数组, 指定每一张图片在 H,W 方向的偏移量。可以取正/负实数, 绝对值不宜太大 (相对原图大小而言)
 - padding: 平移后向空缺区域内填充的数据, 在 MNIST 数据集上应该取 0。
- 返回平移后的图像 (使用双线性插值)。
- 具体实现:
 - 得到新图片中每一个像素对应的原图索引 grid_h, grid_w:
 - 初始化 grid_h, grid_w 数组。
 - 先初始化大小为 [B, C, H, W] 的全零数组 grid_h, grid_w。
 - 用 np.arange() 配合广播机制, 先给 grid_h, grid_w 中填入对应于原图的行号/列号。
 - 具体而言, grid_h 中每一行内部填写相同的行号, grid_w 每一列内部填写相同的列号。
 - 从 grid_h, grid_w 中分别“减去” shift_h, shift_w, 得到新图片中每一个位置对应的原图索引。
 - 调用双线性插值 BiLinearInterpolation(imgs, grid_h, grid_w, padding), 返回。

6.1.3 图像的批量旋转

rotate2D(imgs, theta, padding)

- 参数:
 - theta: 大小为 B 的数组, 指定每一张图片以图像中心为原点, “顺时针”方向的旋转弧度数。可以取正/负实数, 绝对值不应该超过 1.57 (180°)。
 - imgs, padding 同上。
- 返回旋转后的图像 (使用双线性插值)。
- 具体实现:
 - 用和“平移”类似的方法初始化 grid_h, grid_w 数组。
 - 注意需要以图像中心为远点, 额外减去 (H-1)/2 和 (W-1)/2。
 - 计算 grid_h, grid_w 数组指定的坐标点“逆时针”旋转 theta 后的坐标。
 - 注意需要变回原图像的下标, 补加上 (H-1)/2 和 (W-1)/2。
- 调用双线性插值, 返回。

6.1.4 图像的批量缩放

scale2D(imgs, scale, padding)

- 参数:
 - scale: 大小为 B 的数组, 指定每一张图片以图像中心为原点, “放大”的程度。只能取正实数, 取值应该在 1 附近。
 - imgs, padding 同上。
- 返回缩放后的图像 (使用双线性插值)。
- 具体实现:

- 用和“旋转”相同的方法初始化 grid_h, grid_w 数组。
 - 注意仍然需要以图像中心为远点，额外减去 (H-1)/2 和 (W-1)/2。
- 计算 grid_h, grid_w 数组指定的坐标点“缩小” scale 后的坐标。
 - 注意仍然需要变回原图像的下标，补加上 (H-1)/2 和 (W-1)/2。

6.1.5 其它适配

- mynn 目录下的 __init__.py 中增加：


```
from mynn import data_augmentation
```
- 考虑过先对“下标”做完 平移+旋转+缩放 以后，再调用双线性插值 BiLinearInterpolation。但是一方面下标计算很麻烦，另一方面原本的实现的耗时是可接受的，故没有调整。
 - 实际测试时，对 50000 条训练集作用批量平移/旋转/缩放用时约 $3 \times 8 = 24s$ 。

6.1.6 效果可视化

对五张图像分别作用平移/旋转/缩放，效果如下：

第一行是原图，第二行是平移后的结果，第三行是旋转后的结果，第四行是缩放后的结果。



6.2 "数据增强器"类 MyDataAugmentor

`MyDataAugmentor(max_shift_h, max_shift_w, max_angle, scale_range, prob, padding)`

为了方便训练文件中进行数据增强，定义了 MyDataAugmentor 类。

- 初始化方法 `__init__()`
 - 参数：
 - `max_shift_h, max_shift_w`: 平移的最大像素数量
 - `max_angle`: 旋转的最大弧度数
 - `scale_range`: 缩放区间，用可索引的数据结构给出
 - `prob`: 图像被增强的概率
 - `padding`: 图像变换时向空缺区域内填充的数据
 - 将所有参数存入类的私有变量中
- 数据增强

`data_augmentation(imgs)`

根据初始化时提供的参数，对批量图片 `imgs` 进行数据增强。

返回增强后的结果，其元素和 `imgs` 一一对应，可以适配原本的类标签。

返回结果的维数和 `imgs` 相同，该结果不会覆盖 `imgs`。

- 根据 `self.max_shift_h` 等参数，用标准正态采样 `shift_h` 等变量，注意变量范围。
- 使用采样的 `shift_h` 等变量调用 `scale2D`, `rotate2D`, `shift2D` 对数据 `imgs` 进行缩放、旋转、平移变换。
- 由 `self.probs` 生成掩码，将原图像和变换后的图像组合（避免覆盖原数组，使用 `.copy()` 深拷贝），返回。

6.3 静态数据增强

在训练开始前一次性对训练集进行数据增强。

训练过程中使用同一个增强后的训练集，大小和原训练集相同，即增强后的数据会覆盖原训练数据。

- 在 `test_train_CNN.py` 中用 `AUGMENTATION = 'Static'` 指定使用静态数据增强。
- 定义 `runner` 前，定义“数据增强器”，调用 `.data_augmentation(train_imgs)` 对训练数据增强，结果仍然存入 `train_imgs` 中供 `runner` 使用。
 - MLP 模型需要先把 `train_imgs` 转化为图像尺寸，再做数据增强，然后再回到原本的尺寸。
- 由于后续动态数据增强的需求，在 `runner` 中增加了 `data_augmentor` 参数。静态数据增强在不会在训练过程中做数据增强，设置 `runner` 中的 `data_augmentor` 为 `None`。

6.4 动态数据增强

在训练过程的每一次前向传播前，动态对数据进行增强。

每一个 `epoch` 看到的数据是不一样的。

6.4.1 修改 `runner.py`: `RunnerM` 类

- 初始化方法 `__init__()`

- 增加参数 `data_augmentor`，默认为 `None`，将该参数存入 `self.data_augmentor` 中。
数据增强发生在训练过程中，需要提供"数据增强器"的实例作为类的参数。
- 增加参数 `image_shape`，默认为 `None`，仅在 `data_augmentor` 非空时有效，该参数存入 `self.image_shape` 中。
因为数据增强必须对原始图像而不是展平后的图像操作，为了在 MLP 模型中也可以应用动态数据增强，需要知道原始图像的形状。
- 训练 `train()`
取出在每个 batch 的训练集 `train_X` 后，若设定了 `data_augmentor`，则需要对 `train_X` 进行数据增强。
 - 需要先将训练集 `train_X` 变成原始图像的形状，最后再变回去。
 - 先存下原本 `train_X` 的形状。
 - 再根据 `self.image_shape` 的取值重塑 `train_X` 的形状。
 - 对变成图像尺寸的 `train_X` 作用 `self.data_augmentor.data_augmentation()`。
 - `train_X` 恢复到存下的原始形状。
 - 为了防止修改 `train_X` 时一并修改了原始的训练集 `X`，将从 `X` 中提取 `train_X` (一个 batch 的训练集) 的操作改为深拷贝 (使用 `.copy()`)。

6.4.2 修改 test_train_CNN.py

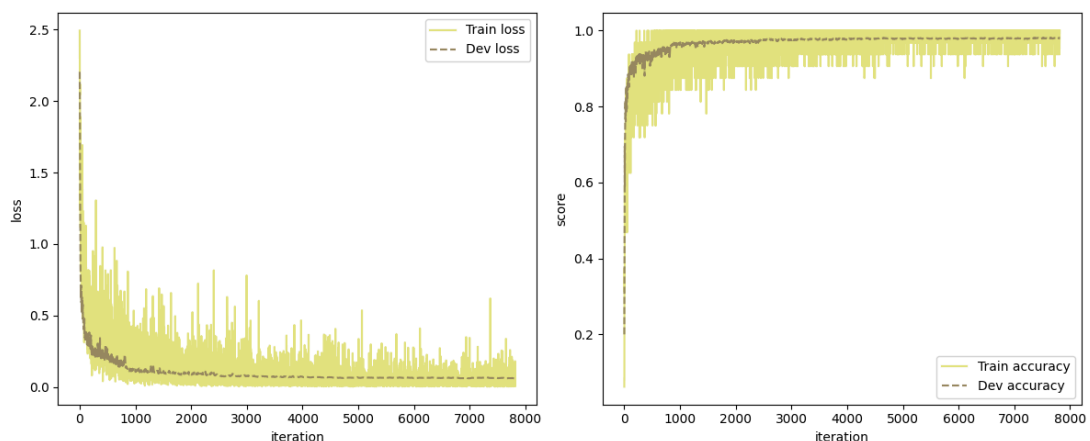
- 在 `test_train_CNN.py` 中用 `AUGMENTATION = 'Dynamic'` 指定使用动态数据增强。
- 定义 `runner` 前，定义"数据增强器"，但不在此处调用 `.data_augmentation(train_imgs)` (在 `runner` 中调用)。
- 定义的"数据增强器"传入 `runner` 初始化的 `data_augmentor` 参数中。

6.5 训练 MLP

沿用上面得到准确率最高的配置: Momentum GD + Kaiming 初始化。

6.5.1 MLP: 使用"静态数据增强"

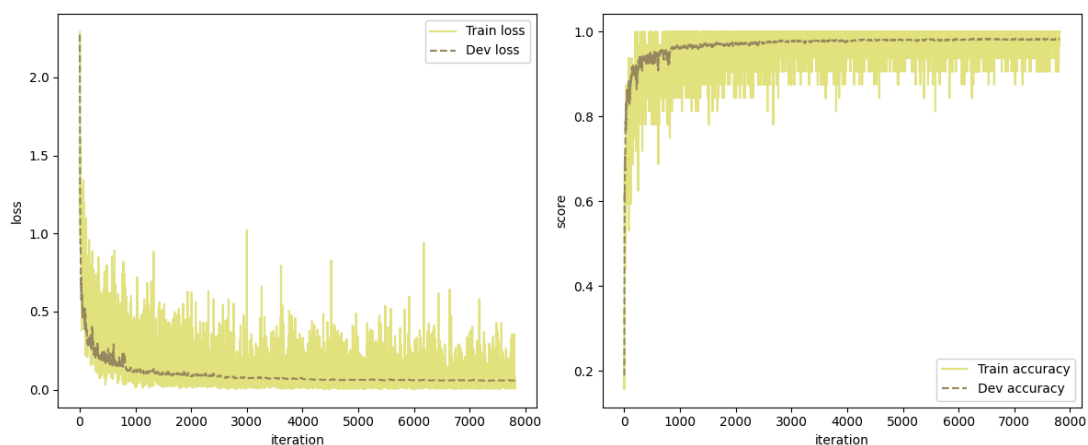
在 `test_train.py` 中设置 `AUGMENTATION = 'Static'`，使用默认参数初始化 `MyDataAugmentor` (暂时设置 `prob = 0.4`)，其余配置和 5.2.1 节相同，训练结果：



在验证集上的准确率约为 0.9802，在测试集上的准确率约为 0.9819。

6.5.2 MLP: 使用"动态数据增强"

在 test_train.py 中设置 AUGMENTATION = 'Dynamic' , 使用默认参数初始化 MyDataAugmentor (除了 prob = 0.4) , 其余配置和 5.2.1 节相同, 训练结果:



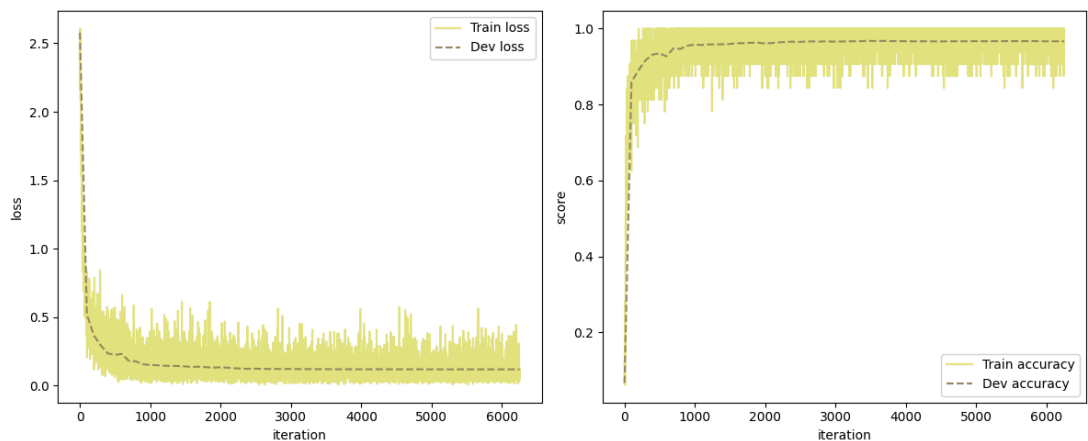
在验证集上的准确率约为 0.9827, 在测试集上的准确率约为 0.9834。

6.6 训练 CNN

沿用上面得到准确率最高的配置: Monent GD + Kaiming 初始化。

6.6.1 CNN: 使用"静态数据增强"

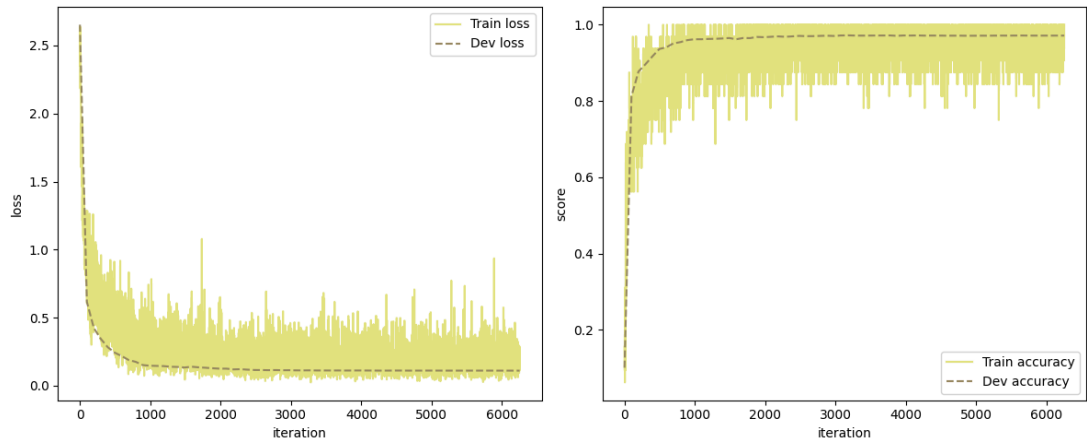
在 test_train_CNN.py 中设置 AUGMENTATION = 'Static' , 使用默认参数初始化 MyDataAugmentor (除了 prob = 0.4) , 其余配置和 5.3.1 节相同, 训练结果:



在验证集上的准确率约为 0.9666, 在测试集上的准确率约为0.9688。

6.6.2 CNN: 使用"动态数据增强"

在 test_train_CNN.py 中设置 AUGMENTATION = 'Dynamic' , 使用默认参数初始化 MyDataAugmentor (除了 prob = 0.4) , 其余配置和 5.3.1 节相同, 训练结果:



验证集上准确率约为 0.9718，测试集上准确率约为 0.9720。

数据增强器的定义

```
data_augmentor = nn.data_augmentation.MyDataAugmentor(max_shift_h=3,
max_shift_w=3, max_angle=0.1, scale_range=(0.9, 1.1), prob=0.4,
padding=0)
```

结论：MLP 和 CNN 应用 Moment GD 和 Kaiming 初始化时，动态数据增强 > 静态数据增强 > 无数据增强。

- 注意到在数据增强变换后，图像看上去变得较为模糊（对比度下降），模型需要做更多努力以分类模糊的图像，所以准确率上升。
- 动态数据增强可以被视为扩大了训练集，模型在训练过程中见到的图像不止 50000 条训练集图像，增强了模型的泛化性。

7. 正则化: Dropout

将每一个神经元（激活函数）的输出以 $1-p$ 概率置零。参照 lec 5 P85 的实现，修改 models.py。

7.1 Dropout 实现

7.1.1 修改 models.py

```
Model_MLP(input_dim, nHidden, output_dim, act_func, lambda_list,
initialization_method, dropout_p)
```

```
Model_CNN(layers, dropout_p)
```

对 Model_MLP 和 Model_CNN 同样修改。

- 初始化 `__init__()`:
 - 增加参数 `dropout_p`，表示单个神经元的输出被保留的概率，默认为 1。
 - 参数 `dropout_p` 被存入 `self.p` 中。
- 前向传播 `forward()`:
 - 获得当前层 `layer` 的输出 `outputs` 后，检查层 `layer` 的类型。
 - 若为激活函数中的一种，则根据 `self.p` 的值生成掩码。掩码中非零元素的值为 $1/\text{self.p}$ ，这样可以避免额外修改评估 `evaluation` 函数。
 - 对输出 `outputs` 作用掩码。

- 用于获取验证集/测试集准确率的 `predict()`:
 - 在验证集/测试集上测试时，应该使用全部权重，`predict()` 和原本的 `forward()` 相同。

注意新增激活函数时需要在 `if` 语句中增加一点代码。

定义 MLP 和 CNN 时需要额外指定 `dropout_p`。

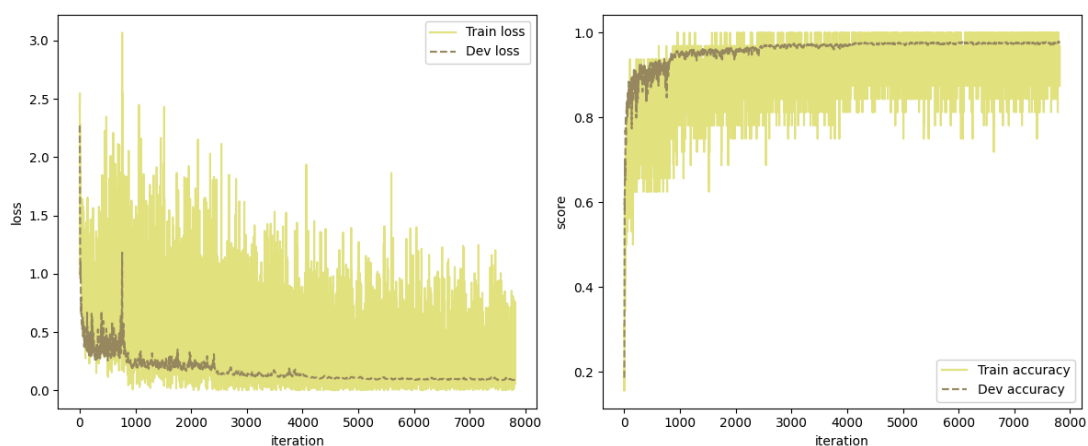
7.1.2 其它修改

涉及到验证集/测试集评估的地方，需要修改 `model.forward()` (即直接调用 `model()`) 为 `model.predict()`。

- `runner.py: evaluate()` 中，修改 `model(test_imgs)` 为 `model.predict(test_imgs)`。
- `test_model.py` 和 `test_model_CNN.py` 中，修改 `model(test_imgs)` 为 `model.predict(test_imgs)`。
 - 其实这里是否修改都是等价的。
 - 因为两个 `test_model` 文件中都会先以默认参数初始化模型，再调用 `model.load_model()` 加载模型覆盖默认参数。
 - 而默认参数设置 `self.p = dropout_p = 1`，此时 `model.forward()` 等价于 `model.predict()`。
 - 考虑到可能会修改 `models.py` 的模型类中 `dropout_p` 的默认值，还是使用 `model.predict(test_imgs)` 代替 `model(test_imgs)`。

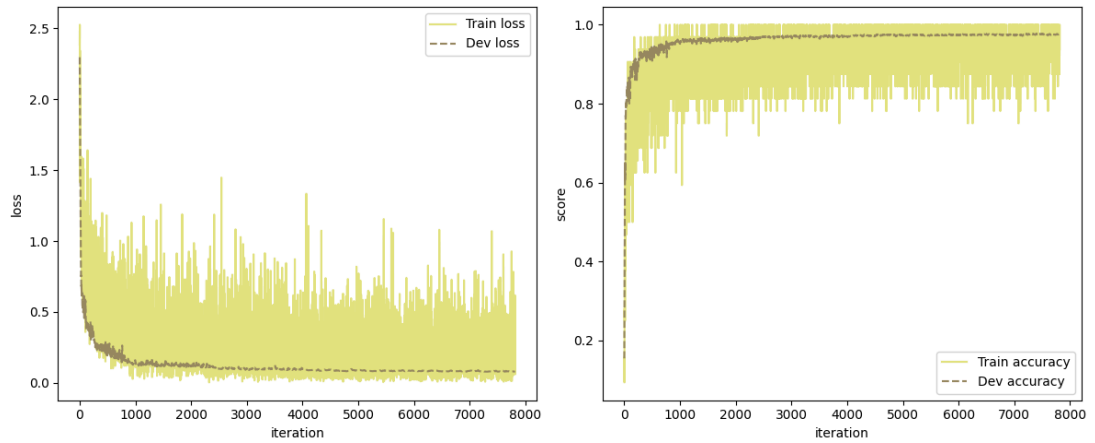
7.2 应用 dropout 训练 MLP

设置 `dropout_p = 0.5`，其余配置和 6.5.2 相同 (Moment GD + Kaiming 初始化 + 动态数据增强 + dropout)。训练结果:



验证集上准确率约为 0.9767，测试集上准确率约为 0.9792。

损失曲线看上去似乎出现了一些震荡，可能是学习率过高的原因，下调学习率至 0.03 (原本为 0.06) 再次训练。训练结果：



验证集上准确率约为 0.9760，测试集上准确率约为 0.9786。

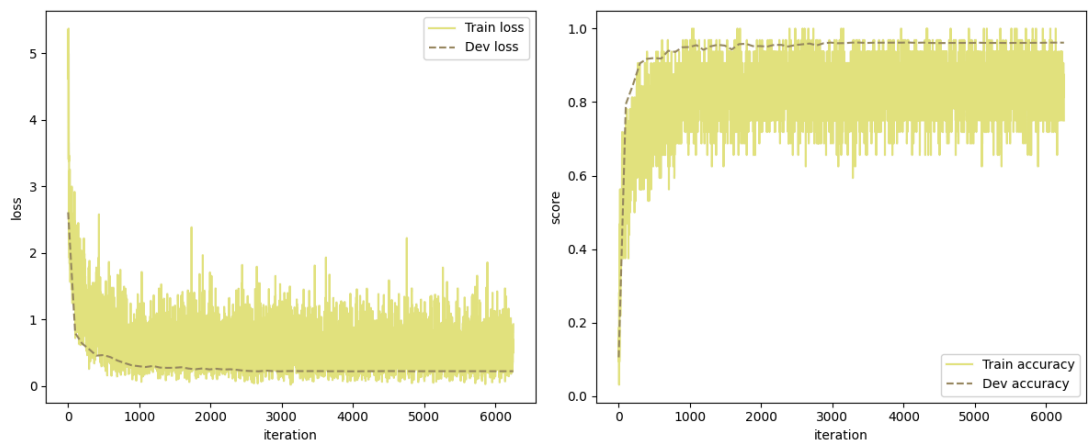
看上去还是学习率设置为 0.06 效果好一点。

加上 *dropout* 后的 *MLP model* 定义

```
linear_model = nn.models.Model_MLP(train_imgs.shape[-1], [600], 10,
    'ReLU', [1e-4, 1e-4], initialization_method='Kaiming', dropout_p=0.5)
```

7.3 应用 dropout 训练 CNN

设置 `dropout_p = 0.5`，其余配置和 6.6.2 相同 (Moment GD + Kaiming 初始化 + 动态数据增强 + dropout)。训练结果:



验证集上准确率约为 0.9617，测试集上准确率约为 0.9617。

加上 *dropout* 后的 *CNN model* 定义

```
cnn_model = nn.models.Model_CNN(layers = [...], dropout_p=0.5)
```

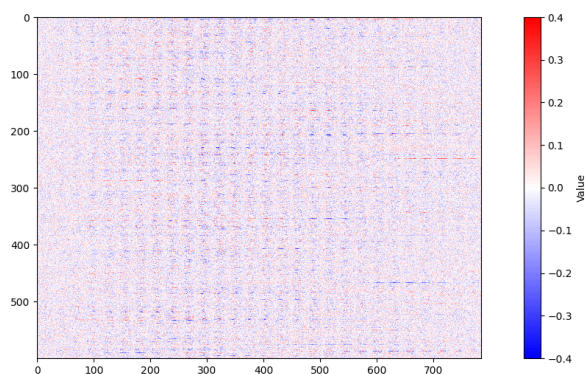
结论：看上去 dropout 对 MLP 和 CNN 的性能并没有提升，可能是因为没有找到合适的超参（学习率过高，或层数太少并不很需要正则化）导致的。

8. 可视化权重

8.1 可视化 MLP 权重

选用先前性能最好的 MLP 模型进行可视化 (Moment GD + Kaiming 初始化 + 动态数据增强)。

8.1.1 MLP 第一层全连接层的权重整体可视化



绘图实现：

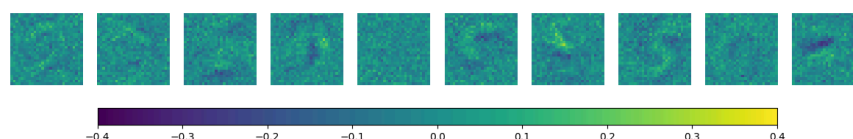
- 先对第一层整体 784×600 的权重作图，确认权重范围大致在 -0.4 至 0.4 之间。
- 为了让图像在报告中不占用太大面积，对权重矩阵转置后绘图。
 - 图像中的“一行”是 784 个数值，可以视为 28×28 的矩阵被拉开后的结果，对应提取出的一个特征。

分析：

- 第一层的权重集中于 -0.4 至 0.4 内，大多数权重值较接近 0。
- 图像中呈现出多处有规律的类似“虚线”的横向结构。
 - 可能因为：横向的一个“线”状结构对应一行，即某一个被提取出的特征。不同“行”专注于提取不同特征，在图中表现为比较明显的横向直线结构。
- “虚线”中较“虚”的位置在各行之间比较统一，使纵向也呈现出一定纹理结构。较为明显的“虚线”的位置大多在每一行的中间附近。
 - 可能因为：大部分特征更关注图像中心而非边缘的像素取值。可以验证每一行中比较“虚”的地方对应的列号在 28 的倍数附近。
 - 全连接层提取一个特征，相当于用两个 784 维的向量（图片和权重中的一行）做内积。在图片原始尺寸的视角上，就是用 28×28 大小的张量和图片本身（同样是 28×28 的张量）做内积。一“行”对应的 784 个数字，可以被视为一个 28×28 的“卷积核”按行展开的结果。

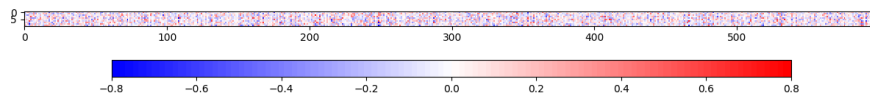
8.1.2 MLP 第一层全连接层的权重以矩阵形式可视化

按照上述思路，以矩阵的形式可视化上述图像中的“行”。为了更好地捕捉每一个矩阵的结构，这里更换了颜色映射方式：



可以看到图中似乎呈现出一些有规律的模式。好像可以看到类似数字 9, 8, 5, 0 的形状。

8.1.3 MLP 第二层全连接层的权重整体可视化



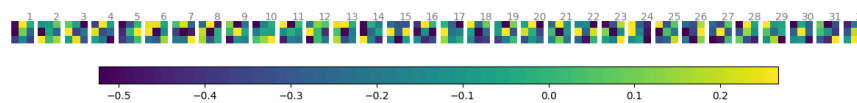
第二层的权重集中于 -0.8 至 0.8 内，看上去难以发现规律。

8.2 可视化 CNN 权重

选用先前性能最好的 CNN 模型进行可视化 (Moment GD + Kaiming 初始化 + 动态数据增强)。

8.2.1 CNN 卷积层1权重可视化

第一层中有 32 个 $1 \times 3 \times 3$ 卷积核，可视化结果：

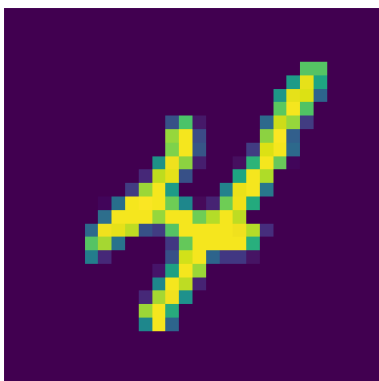


分析：

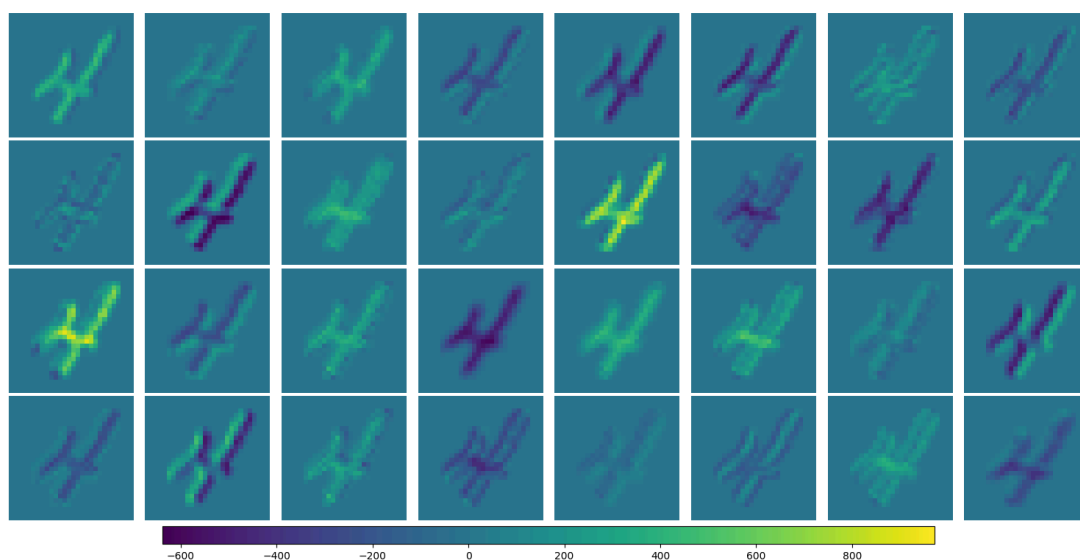
- 第一层卷积核的取值集中于 -0.5 到 0.25 之间。
- 第 1, 24 号卷积核看上去具有纵向条纹，应该用于提取纵向边缘。
- 第 3, 4, 10 号卷积核看上去呈现对角线条纹，应该用于提取倾斜 45° 的纹理。
- 第 27, 30 号卷积核的权重中心大周边小，可能更关注图像较为中心区域的特征。

8.2.2 CNN 图片作用卷积层1后的特征可视化

从训练集中选择了一张图片：



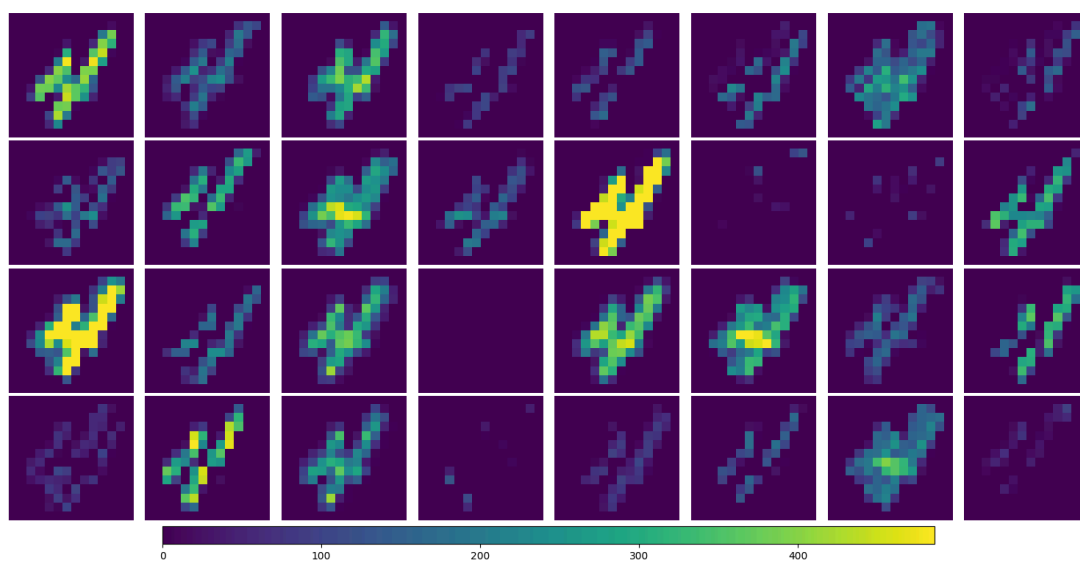
单张图片 $1 \times 1 \times 28 \times 28$ 作用卷积层1 ($32 \times 1 \times 3 \times 3$) 后, 得到 $1 \times 32 \times 14 \times 14$ 的特征图:



可以看到不同卷积核已经初步提取了图片的不同特征。(例如 10 号卷积核提取了图片的倾斜纹理, 16 号卷积核提取了边缘特征)

8.2.3 CNN 图片作用激活函数1和池化层1后的特征可视化

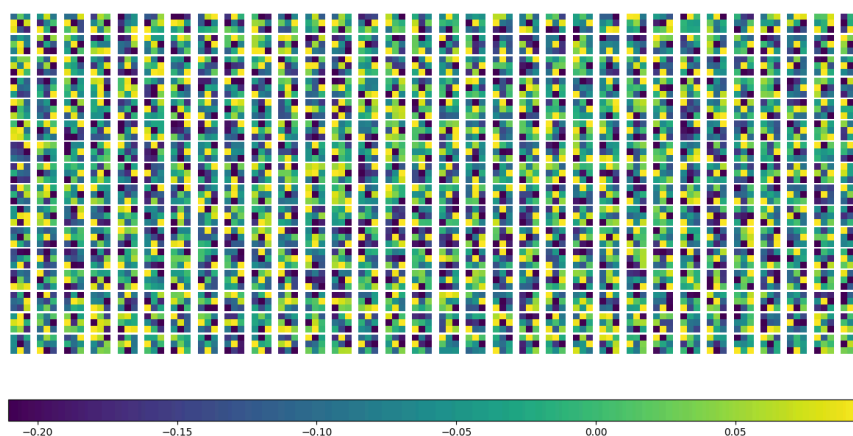
$1 \times 32 \times 14 \times 14$ 的特征图作用 ReLU 和 Pooling 后, 得到 $1 \times 32 \times 7 \times 7$ 的特征图:



看上去不同图片的最大值差异比较大, 有些图上的值普遍比较小, 没有显示出来。特征进一步显现。(例如 10 号图像提取了倾斜的特征, 12 号图像提取了水平的特征)

8.2.4 CNN 卷积层2权重可视化

第二层中有 64 个 $32 \times 3 \times 3$ 卷积核，下面仅可视化 8 个 $32 \times 3 \times 3$ 的卷积核：

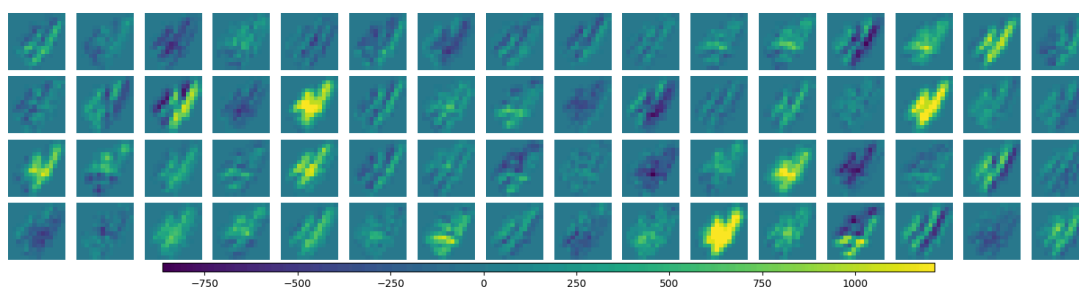


分析：

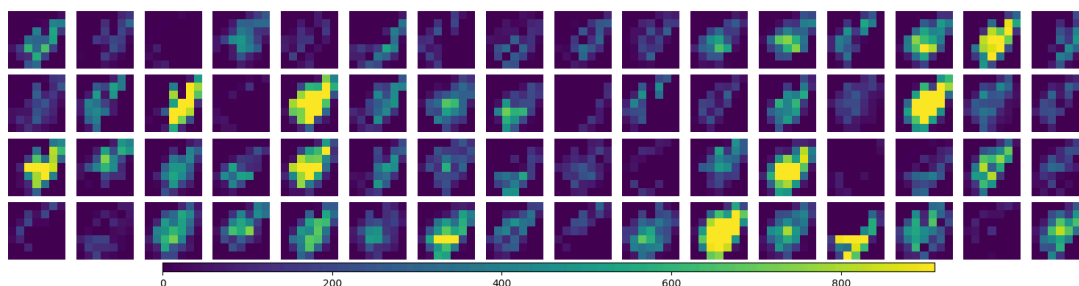
- 第二层卷积核的取值集中于 -0.2 到 0.05 之间。
- 看上去比较难以发现行与行之间的明显差异。

8.2.5 CNN 图片作用卷积层2、激活函数2和池化层2后的特征可视化

$1 \times 32 \times 7 \times 7$ 的特征图作用卷积层2 ($64 \times 32 \times 3 \times 3$) 后，得到 $1 \times 64 \times 14 \times 14$ 的特征图：



$1 \times 64 \times 14 \times 14$ 的特征图作用 ReLU 和 Pooling 后，得到 $1 \times 64 \times 7 \times 7$ 的特征图：

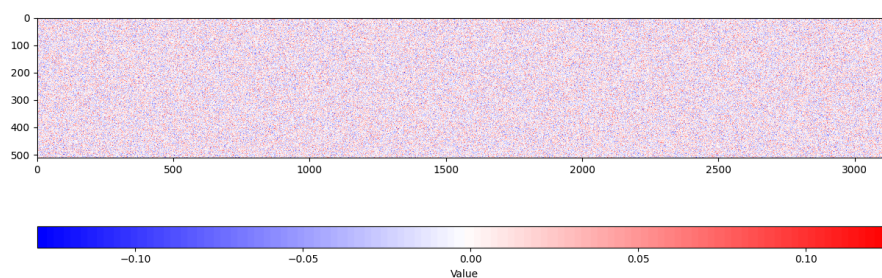


看上去高层特征比底层特征更抽象，缺乏可解释模式。

同时有不少特征看上去非常相似，可能是因为 MNIST 数据集比较简单，数据的总体差异比较小。

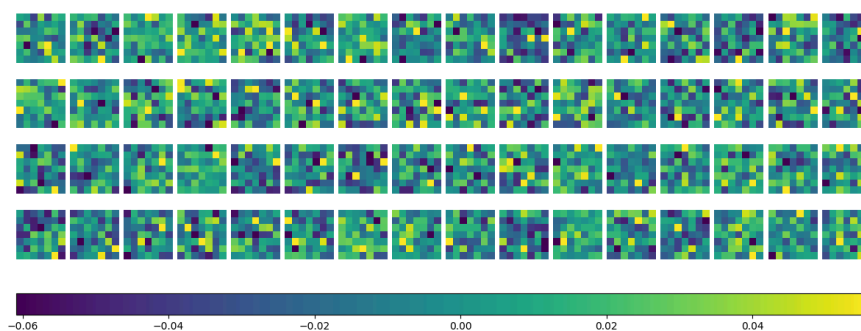
8.2.6 CNN 全连接层1权重可视化

第三层维数 3136×512 ，转置后可可视化：



第三层权重集中于 -0.1 到 0.1 之间，但看上去分布接近随机，没有明显规律。

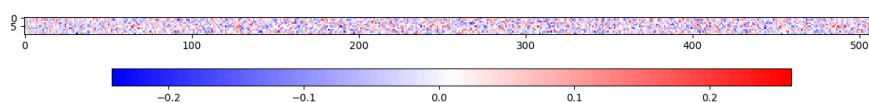
尝试将 3136×512 中第一行的取值转换成 64 个 7×7 的矩阵 (64个通道，每一个通道对一张 7×7 的特征图作用) 进行可视化：



看上去同样比较抽象，缺乏规律。

8.2.7 CNN 全连接层2权重可视化

第四层维数 512×10 ，转置后可可视化：



第四层权重集中于 -0.2 到 0.2 之间，同样缺乏规律。

结论

- Momentum GD + Kaiming初始化 + 动态数据增强 可以取得最佳效果 (MLP 98.34%，CNN 97.20%)，进一步增加数据增强中图片被增强的概率可能可以得到更好结果。
- 数据增强和参数初始化对模型性能影响显著。
- CNN 性能并没有显著优于 MLP，可能是因为超参选择问题，或 CNN 的网络结构过于简单。

	MLP	CNN
SGD + StdNormal	0.9360	0.8420
SGD + Kaiming	0.9534	0.9007
Moment GD + StdNormal	0.9278	-
Moment GD + Kaiming	0.9795	0.9639
静态数据增强	0.9819	0.9688
动态数据增强	0.9834	0.9720
Dropout	0.9792	0.9786

其中，静态数据增强/动态数据增强 在 Moment GD + Kaiming 的配置下完成；
Dropout 在 Moment GD + Kaiming + 动态数据增强 的配置下完成。
其余配置详见报告和代码文件。